

EQuADiSE: Efficient Quantum-safe Adaptive Distributed Symmetric-key Encryption

Sayani Sinha¹, Sikhar Patranabis², and Debdeep Mukhopadhyay¹

¹IIT Kharagpur, India

²IBM Research India

June 17, 2026

Abstract

Distributed symmetric-key encryption (DiSE), introduced in CCS' 18, enables threshold versions of traditional (symmetric-key) authenticated encryption. In DiSE, the long-term master secret key is secret-shared among multiple parties following a threshold access structure, and both encryption and decryption are performed in a distributed manner. An adaptively secure DiSE, introduced in INDOCRYPT' 20 tolerates adaptive corruptions of the key-holding parties for arbitrary thresholds, while simultaneously retaining efficient encryption and decryption. Unfortunately, all existing instances of adaptively secure DiSE are either quantum-unsafe (due to their inherent-reliance on discrete log-hard groups), or incur exponential (in the number of parties) online overheads for encryption/decryption.

In this paper, we present EQuADiSE – the first practically efficient, adaptively secure, and plausibly post-quantum secure construction of DiSE that incurs linear (in the number of parties) encryption/decryption overheads. We prove the adaptive security of EQuADiSE in the Quantum Random Oracle model (QROM) based on the Module Learning with Rounding (MLWR) assumption. As a core building block of independent interest, we introduce an MLWR-based distributed pseudorandom function (DPRF) that enjoys adaptive security in the QROM and practically outperforms *all* existing adaptively secure DPRF constructions in terms of online evaluation time.

We present a prototype implementation of EQuADiSE and evaluate its performance. Our experiments demonstrate that EQuADiSE achieves higher online throughput (number of encryptions/decryptions per second) than *all* prior realizations of DiSE, *including* quantum-unsafe realizations based on discrete log-hard groups.

Contents

1	Introduction	3
1.1	Our Contributions	4
1.2	Technical Overview	5
2	Preliminaries	7
2.1	Cryptographic Background	7
2.2	Distributed PRF (DPRF)	8
2.3	Lattice-based Computational Assumptions	10
2.4	Some Basic Lemmata	10
3	Adaptively Secure DPRF from MLWR	11
3.1	Background: LWR-based Non-Adaptive DPRF [SPM24]	11
3.2	Our Module-LWR based DPRF	12
3.3	Adaptive Security of MLWR-based DPRF	14
3.3.1	Adaptive Security of LWR-based DPRF [SPM24] in the ROM	14
3.3.2	Adaptive Security of LWR-based DPRF [SPM24] in the QROM	19
3.3.3	Adaptive Security of MLWR-based DPRF in the QROM	20
3.4	Choice of Parameters for (M)LWR-based Adaptive DPRF	20
4	EQuADiSE	21
4.1	Construction Overview	21
5	Experimental Evaluation	22
6	Discussion and Future Work	26
A	Known DPRF Constructions	29
B	(t, T)-Threshold Linear Integer Secret Sharing	30

1 Introduction

Threshold cryptography [Des94] enables protecting long-term secret keys by secret-sharing such keys among multiple parties. Any attacker corrupting upto a threshold number of these parties (information-theoretically) fails to recover the secret keys. Threshold cryptography serves as the foundation for several advanced cryptographic primitives such as threshold fully-homomorphic encryption [BGG⁺18], secure multiparty computation [AJLA⁺12], function secret sharing [BGI15], and functional encryption [LT⁺19]. Traditionally, threshold cryptography considers two kinds of corruption models – *static*, where the adversary corrupts a fixed subset of the key-holding parties at the beginning of time, and *adaptive*, where the adversary adaptively chooses which key-holding parties to corrupt over time. For many practical applications, security against adaptive corruptions is more desirable.

Distributed Symmetric-Key Encryption. The study of threshold authenticated symmetric-key encryption was introduced in [AMMR18]. The authors of [AMMR18] introduced a primitive called distributed symmetric-key encryption (DiSE). In DiSE, the master secret key is secret-shared among multiple parties following a threshold access structure, and both encryption and decryption are performed in a distributed manner. DiSE enables protecting long-term keys in many different applications, such as protecting data at rest, authenticating clients on enterprise networks, and securing data and payments on IoT devices.

Informally speaking, the DiSE scheme introduced in [AMMR18] (followed by several other optimizations of DiSE [CGMS21, ADL⁺22, DMSS24, MGB⁺24]) uses the following blueprint. Consider the classic construction of symmetric-key encryption (SKE) from a pseudorandom function (PRF) where the encryption of a message m under a secret key k is given by

$$\text{Enc}(k, m) = (r, F_k(r) \oplus m),$$

where F_k is a PRF and r is a randomly sampled PRF input. The core idea of [AMMR18] is to distribute the encryption/decryption procedure by replacing the PRF F_k by a distributed PRF (DPRF) [NPR99]. A DPRF allows securely evaluating $F_k(\cdot)$ on any input in a distributed manner, while the secret key k is secret-shared across multiple parties following a threshold access structure. There exist known constructions of DPRFs from purely symmetric-key assumptions such as any PRF [NPR99], as well constructions from number-theoretic assumptions such as the decisional Diffie-Hellman (DDH) assumption [NPR99] and lattice-based assumptions such as Learning with Errors (LWE) [LST21], Learning with Rounding (LWR) [SPM24]. Following the blueprint of [AMMR18], these instantiations of DPRF yields DiSE schemes with varying efficiency properties.

A DiSE scheme is said to be *ideally efficient* if: (i) the key storage required by each party is linear in the number of key-holding parties, and (ii) the online evaluation time for encryption/decryption is linear in the number of key-holding parties. In practice, the second requirement is often more important, particularly for applications (such as authenticating clients on enterprise networks) where ensuring fast online turnaround time is crucial.

Adaptive Security. A DiSE scheme is said to be *adaptively* secure if it tolerates adaptive corruptions of the key-holding parties for arbitrary thresholds, while simultaneously retaining efficient encryption and decryption. Among the known instantiations of DiSE, the one based on purely symmetric-key realizations of DPRF is adaptively secure, but is asymptotically inefficient since it incurs *both* exponential key storage *and* exponential online overheads for encryption/decryption. The only efficient yet adaptively secure DiSE scheme is based on an adaptive DPRF from the DDH assumption [Muk20]. While this construction is ideally efficient and plausibly secure against classical attackers, it is unsafe against known quantum algorithms for discrete log [Sho94].

Post-Quantum Adaptive DiSE. In this paper, we are interested in building practically efficient, adaptively secure, and plausibly post-quantum instances of DiSE. Note that such a construction follows immediately from a DPRF satisfying the same set of properties simultaneously. Unfortunately, no existing post-quantum secure DPRF simultaneously achieves adaptive security and the desired notions of efficiency as outlined above. We summarize known instances of plausibly post-quantum DPRFs and their properties below.

DPRF from any PRF. The known construction of DPRF from any PRF [NPR99] is adaptively secure,

DPRFs	Key Share Size	Partial Evaluation Overheads	Total Evaluation Overheads	Communication Overheads	Post-Quantum?	Adaptively Secure?
AES(128)-based (combinatorial) [AMMR18]	$O(\binom{t-1}{t-1})$	$O(\binom{t-1}{t-1})$	$O(\binom{t-1}{t-1})$	$O(\binom{t-1}{t-1})$	No	Yes (small t, T)
DDH-based [Muk20]	$O(1)$	$O(1)$	$O(t)$	$O(t)$	No	Yes
ThFHE-based [BGG ⁺ 18]	$O(\binom{t-1}{t-1})$	$O(1)$	$O(t)$	$O(t)$	Yes	No
LWE-based [LST21]	$O(\binom{t-1}{t-1})$	$O(\lambda t)$	$O(\lambda t)$	$O(t)$	Yes	Yes
LWR-based [SPM24]	$O(\binom{t-1}{t-1})$	$O(1)$	$O(t)$	$O(t)$	Yes	No
MLWR-based (Ours)	$O(\binom{t-1}{t-1})$	$O(1)$	$O(t)$	$O(t)$	Yes	Yes

Table 1: Asymptotic comparison of DPRFs for λ -bit inputs (λ being the security parameter). Note that for the adaptively secure LWE-based DPRF from [LST21], the factor of λ in the overheads for partial and total evaluation reflects the requirement of $O(\lambda)$ matrix multiplications for every partial evaluation. In comparison, our DPRF requires a constant number of polynomial multiplications for each partial evaluation.

but incurs *both* exponential key storage *and* exponential online overheads for encryption/decryption. This essentially captures schemes which perform a multi-party computation (MPC)-based evaluation of any PRF.

DPRF from Threshold FHE. One can build a DPRF in a generic manner from any PRF and a threshold fully-homomorphic encryption (FHE)-based *universal thresholdizer* [BGG⁺18]. Based on known instances of threshold FHE [BGG⁺18, CSS⁺22, DDEK⁺23, BS23], such a construction incurs exponential key storage, but linear online overheads for encryption/decryption. Unfortunately, all known threshold FHE schemes are only statically secure, and hence the resulting DPRF is not adaptively secure. Additionally, the universal thresholdization approach requires non black-box evaluation of the PRF under the hood of FHE, which renders the resulting DPRF practically inefficient, and unsuitable for applications of DiSE.

DPRF from LWE [LST21]. An adaptively secure DPRF from the LWE assumption was presented in [LST21]. This scheme incurs exponential key storage, but linear overheads for online evaluation. Unfortunately, this scheme requires a *super-polynomial modulus-to-noise ratio*, and involves a log-depth (in the size of the input) evaluation circuit (aka Naor-Reingold [NR97]) consisting of several matrix multiplications. Hence, this scheme does not yield a practically efficient and scalable realization of DiSE.

DPRF from LWR. Recently, the authors of [SPM24] proposed a practically efficient DPRF construction based on LWR and a corresponding instantiation of DiSE, called PQDiSE. PQDiSE incurs exponential key storage, but linear online overheads for encryption/decryption. Unfortunately, their construction is only proven to be statically secure. While adaptive security may be achieved by naïvely guessing the corrupted set a priori, this leads to an exponential blow-up in the online overheads for encryption/decryption, thus compromising on efficiency.

In this paper, we ask the following question:

Can we design a practically efficient, adaptively secure, and plausibly post-quantum instance of DiSE?

More concretely, we investigate the possibility of designing a practically efficient and adaptively secure DPRF from lattice-based hardness assumptions, with linear online evaluation overheads, while avoiding the requirement of a super-polynomial modulus-to-noise ratio as in [LST21].

1.1 Our Contributions

We answer the above question in the affirmative. Our contributions are as summarized below.

Adaptive DPRF from Module LWR. We introduce an adaptively secure DPRF from the Module Learning with Rounding (MLWR) assumption [BGV14, BGM⁺16, DKSRV18] with polynomial modulus-to-noise ratio. We prove the adaptive security of our construction in the Quantum Random Oracle model (QROM). Our construction supports arbitrary threshold access structures and incurs exponential key storage but linear online evaluation overheads, thus matching the asymptotic efficiency of the best lattice-based non-adaptive DPRFs. In terms of online evaluation time, our construction outperforms *all* existing adaptively secure DPRF constructions.

Table 1 compares our proposed DPRF against existing DPRFs for λ -bit inputs (λ being the security parameter). See Appendix A for a detailed summary of these DPRFs. Except [LST21], all prior constructions are either quantum-unsafe, or lack adaptive security, or suffer from inefficient online evaluation. In the case of [LST21], the factor of λ in the evaluation overheads reflects that each partial evaluation requires $O(\lambda)$ matrix multiplications, where the matrix dimensions are similar to the degree of a polynomial in our MLWR-based DPRF. In contrast, our MLWR-based DPRF only requires a constant number of polynomial multiplications for each partial evaluation, and is significantly more efficient in practice.

EQuADiSE. Our proposed DPRF yields a realization of DiSE, which we call EQuADiSE (**E**fficient and **Q**uantum-safe **A**daptive **D**istributed **S**ymmetric-key **E**ncryption). EQuADiSE is the first practically efficient, adaptively secure, and plausibly post-quantum instance of DiSE that incurs linear (in the number of parties) encryption/decryption overheads.

Implementation and Evaluation. In Section 5, we experimentally evaluate the practical efficiency of our proposed MLWR-based DPRF and EQuADiSE, and compare these against all of the DPRFs and the corresponding realizations of DiSE from Table 1. Our results show that our MLWR-based DPRF is faster than all other DPRFs (including the ones based on AES-128 and DDH) in terms of online evaluation time, particularly for larger values of (t, T) (where T denotes the total number of parties and t denotes the threshold). The naïve distributed construction of AES-based DPRF is essentially equivalent to a generic MPC-based technique of evaluating AES (which is viewed as a PRF). Thus, our construction is more efficient than a generic MPC-based solution of DPRF. These efficiency benefits also translate to EQuADiSE, which achieves higher online throughput than *all* prior realizations of DiSE, and scales smoothly to larger values of (t, T) . Our implementations of DPRFs and EQuADiSE are available here¹.

1.2 Technical Overview

In this section, we present an overview of the techniques underlying our adaptively secure MLWR-based DPRF, which forms the core of EQuADiSE.

PRF from LWR. The starting point for our proposed DPRF is a simple *weak* PRF $F : \mathbb{Z}_q^n \times \mathbb{Z}_q^n \rightarrow \mathbb{Z}_p$ for appropriately chosen moduli q and p such that $p < q$:

$$F(\mathbf{k} \in \mathbb{Z}_q^n, \mathbf{y} \in \mathbb{Z}_q^n) = \lfloor \langle \mathbf{y}, \mathbf{k} \rangle \rfloor_p,$$

where the operator $\lfloor \cdot \rfloor_p$ takes as input an element in $\mathbb{Z}_q$, and rounds it to the nearest integer in $\mathbb{Z}_p$ (in other words, effectively drops the least significant $(\log q - \log p)$ bits). The above is, by definition, a weak PRF under the LWR assumption, for appropriately chosen q, p . Recall that a weak PRF only satisfies pseudorandomness for uniformly sampled inputs. However, one can turn the above weak PRF into a full-fledged PRF in a straightforward manner by applying a random oracle on the input. While our formal treatment in Section 3 considers the full-fledged random oracle-based PRF, we focus on the weak PRF setting in the overview for ease of exposition. This setting suffices to provide a flavor of our core techniques.

Non-Adaptive DPRF from LWR [SPM24]. In [SPM24], it was shown that the above PRF can be extended to a (t, T) -DPRF for arbitrary (reconstruction and privacy) threshold $t < T$ with security against *static corruptions*. Their approach was to secret share the key $\mathbf{k}$ among T parties using *Linear Integer Secret Sharing* (LISS) [DT06] in the setup phase, such that any subset of t parties can linearly reconstruct the key. Distributed evaluation of the PRF on an input $\mathbf{y}$ is achieved by a t -sized subset $\mathcal{S}$ of parties as follows: first each party $P_i \in \mathcal{S}$ uses its key share $\mathbf{k}_i$ to compute $z_i = \lfloor \langle \mathbf{y}, \mathbf{k}_i \rangle \rfloor_{q_1}$, where q_1 is an additional intermediate modulus such that $p < q_1 < q$. Finally, the evaluator obtains the final output as $z^* = \lfloor f(\{z_i\}_{P_i \in \mathcal{S}}) \rfloor_p$, where f is some linear function of the partial evaluations $\{z_i\}$ (see Section 3.1 for a more formal exposition).

The authors of [SPM24] proved that the above construction is a (weak) DPRF that satisfies both consistency and security against static corruptions of up to $(t-1)$ parties for appropriately chosen $q, q_1, p = \text{poly}(\lambda)$ (λ being

¹<https://github.com/SayaniSinha97/EQuADiSE>

the security parameter) assuming the hardness of LWR for polynomial modulus-to-modulus ratio [BGM⁺16]. At a high level, their proof strategy crucially uses the knowledge of the corrupt set of parties and the corresponding key shares (in the case of static corruptions, this is known at setup) to construct a simulator that simulates an honest party's partial evaluation on any uniformly random input.

Adaptive Security of [SPM24] in ROM. Our first contribution is in proving that the above DPRF is, in fact, secure against *adaptive corruptions* of up to $(t - 1)$ parties for appropriately chosen $q, q_1, p = \text{poly}(\lambda)$, while relying on the same assumption of the hardness of LWR for polynomial modulus-to-modulus ratio. Note that, in the case of adaptive corruptions, the set of corrupt parties is not known from beforehand, and hence, one cannot follow the simulation strategy from [SPM24] of using the knowledge of the corrupt parties' secret shares to simulate the honest party's partial evaluations. In particular, consider a scenario where the adversary adaptively corrupts an honest party P_i and learns its secret share. In this case, all simulated partial evaluations corresponding to P_i with respect to prior queries must be consistent with the revealed secret share once the adversary corrupts P_i . It is not immediately clear how to do this since the simulator does not learn P_i 's key share until the adversary chooses to corrupt it.

To address this, we present an alternative proof strategy that relies on the programmability of the random oracle. At a high level, we show that the simulator is able to perfectly simulate all the responses to the adversary's queries without knowing any of the actual key shares. This effectively implies that no meaningful information about the underlying secret shares is leaked to the adversary through the query-response phase in the real world. To achieve this, the simulator samples one uniform random matrix $\mathbf{A}$ over $\mathbb{Z}_q^{n \times m}$, and another uniform random matrix $\mathbf{R}$ over $\mathbb{Z}_q^{n \times n}$. Let $\mathcal{H}(\cdot)$ be a random oracle that is expected to return a uniformly random $\mathbf{y}$ over $\mathbb{Z}_q^n$ for any query input x .

- For each new query input x , the random oracle response $\mathcal{H}(x)$ is simulated with $(\mathbf{R} \cdot \mathbf{A}) \cdot \mathbf{s}$ for a newly sampled $\mathbf{s} \xleftarrow{\$} \{0, 1\}^m$. The value $\mathcal{H}(x) = (\mathbf{R} \cdot \mathbf{A}) \cdot \mathbf{s}$ represents a random subset-sum of m columns of $(\mathbf{R} \cdot \mathbf{A})$, and due to the leftover hash lemma [Reg09], $\mathcal{H}(x)$ is uniformly random over $\mathbb{Z}_q^n$ for sufficiently large m .
- The response to the partial evaluation query for input x with respect to an uncompromised share $\mathbf{k}_j$ is simulated without using the knowledge of $\mathbf{k}_j$ at all. Basically, a uniform random $\mathbf{k}'_j \xleftarrow{\$} \mathbb{Z}_q^n$ is sampled and the simulated response is $\lfloor \langle \mathbf{A} \cdot \mathbf{s}, \mathbf{k}'_j \rangle \rfloor_{q_1}$. The value $\mathbf{s}$ corresponds to the input x , as specified during a random oracle query for input x . Since $\mathbf{A} \cdot \mathbf{s}$ is uniformly random over $\mathbb{Z}_q^n$ due to the leftover hash lemma [Reg09] and so is $\mathbf{k}'_j$, the distribution of the simulated partial evaluations exactly matches with that of the real ones.
- At a later point in the game, when it is time to reveal the share $\mathbf{k}_j$ to the adversary, (i.e., the adversary compromises $\mathbf{k}_j$), the simulator cheats the adversary by revealing $\mathbf{k}''_j$ instead of $\mathbf{k}_j$. Here, $\mathbf{k}''_j$ is computed as $\mathbf{R}^{-1} \cdot \mathbf{k}'_j$. It is important to note that,

$$\lfloor \langle \mathcal{H}(x), \mathbf{k}''_j \rangle \rfloor_{q_1} = \lfloor \langle \mathbf{A} \cdot \mathbf{s}, \mathbf{k}'_j \rangle \rfloor_{q_1},$$

which implies that the simulated response to the corruption query ($\mathbf{k}''_j$) remains consistent with the previous responses to partial evaluation queries corresponding to $\mathbf{k}_j$, from the point of view of the adversary.

Clearly, for a feasible computation of $\mathbf{k}''_j$, $\mathbf{R}$ must be invertible modulo q . This can be taken care of during the setup of the random oracle while sampling a suitable $\mathbf{R}$. See Section 3.3.1 for the detailed proof techniques.

Security in the Quantum Random Oracle Model. In practice, a random oracle is instantiated by a fixed function, typically a cryptographic hash function. Hence, post-quantum security analyses in the random oracle model should take into consideration adversaries with quantum oracle access [BDF⁺11]. However, the LWR-based DPRF in [SPM24] proves security only against adversaries with classical access. This notion is weaker, since quantum adversaries can query the oracle in superposition, thereby accessing exponentially many inputs per query and potentially extracting more information than any polynomial number of classical

queries allows. We show that our proof of adaptive security for the DPRF construction of [SPM24] holds even in the quantum random oracle model (QROM), while facilitating the adversary with negligible amount of additional distinguishing advantage. See Section 3.3.1 for the detailed proof techniques.

Adaptation to Module LWR. We present a more practically efficient variant of the above-discussed DPRF by switching from the LWR assumption over integer lattices to the Module LWR (MLWR) assumption over module lattices. We note that several NIST standards, notably ML-KEM¹ and ML-DSA²) rely on Module LWE/LWR for better efficiency as compared to their LWE/LWR-based counterparts, and we are not aware of any attacks that perform substantially better against these assumptions as compared to LWE/LWR. Our starting point is the natural adaptation of the LWR-based weak PRF outlined above to the setting of MLWR:

$$F(\mathbf{K} \in \mathcal{R}_q^d, \mathbf{X} \in \mathcal{R}_q^d) = \lfloor \mathbf{X} \cdot \mathbf{K} \rfloor_p \in \mathcal{R}_p$$

where $\mathcal{R}_q$ and $\mathcal{R}_p$ are appropriately chosen polynomial rings, p is an appropriate rounding modulus, and d is the module rank. We then apply the same two-step rounding technique to extend this weak PRF into a weak DPRF. To establish the adaptive security of the MLWR-based DPRF, we adapt our techniques for proving the adaptive security of the LWR-based DPRF with only one key modification, namely, the use of the regularity lemma from [LPR13] instead of the leftover hash lemma [Reg09] to accommodate the module lattice framework. We refer to Section 3.2 for the detailed construction, Section 3.3.3 for the proof of adaptive security, and Section 3.4 for a discussion on concrete parameter choices.

2 Preliminaries

In this section, we present notations and preliminary background material. See Appendix B for additional preliminaries on threshold secret sharing.

Notations. We use $x \leftarrow \mathcal{X}$ to denote that x is sampled from the distribution $\mathcal{X}$, and $x \xleftarrow{\$} X$ to denote that x is sampled uniformly at random from the set X . We denote the cardinality of a set S as $|S|$. We use $[n]$ for some $n \in \mathbb{N}$ to denote the set $\{1, \dots, n\}$.

For a security parameter $\lambda \in \mathbb{N}$, we use $\text{negl}(\lambda)$ and $\text{poly}(\lambda)$ to denote any generic (unspecified) negligible and polynomial function of λ , respectively. We use the notations $\approx_c$ (resp., $\approx_s$) to denote that the computational (resp., statistical) distance between two distributions is negligible in the security parameter λ .

For $q, N \in \mathbb{N}$, $\mathcal{R}_{N,q} = \mathbb{Z}_q[x]/(x^N + 1)$ denotes a ring of polynomials with coefficients in $\mathbb{Z}_q$ and degree at most $(N - 1)$. We use bold upper case symbols (e.g., $\mathbf{A}$) to denote a vector of polynomials over $\mathcal{R}_{N,q}$.

For a pair of polynomials $A, B \in \mathcal{R}_{N,q}$, $(A \cdot B) \in \mathcal{R}_{N,q}$ denotes their polynomial multiplication modulo $(x^N + 1)$. Analogously, for a pair of vectors of polynomials $\mathbf{A}, \mathbf{B} \in \mathcal{R}_{N,q}^d$, $\mathbf{A} \cdot \mathbf{B}$ denotes the inner product $\sum_{i=1}^d \mathbf{A}_i \mathbf{B}_i \pmod{(x^N + 1)} \in \mathcal{R}_{N,q}$.

For any $y \in \mathbb{Z}_q$, the round-off operation, denoted by $\lfloor y \rfloor_p$ gives the nearest integral value of $(y \cdot \frac{p}{q})$ in $\mathbb{Z}_p$; in particular, if $(y \cdot \frac{p}{q})$ has a fractional part exactly equal to 0.5, we choose to always round it down to $\lfloor y \cdot \frac{p}{q} \rfloor$ to avoid ambiguity. We can apply the round-off operator to vectors, matrices and polynomials as well to denote element-wise/coefficient-wise round-off operation.

2.1 Cryptographic Background

In this section, we recall some basic cryptographic primitives.

¹<https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.203.pdf>

²<https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.204.pdf>

Pseudorandom Function (PRF). We recall the formal definition of a pseudorandom function (PRF). Let $\lambda \in \mathbb{N}$ be a security parameter. Let $\mathcal{K}, \mathcal{X}$ and $\mathcal{Y}$ be sets parameterized by λ . Let $F : \mathcal{K} \times \mathcal{X} \rightarrow \mathcal{Y}$ be a family of keyed functions and $\mathcal{F} = \{g \mid g : \mathcal{X} \rightarrow \mathcal{Y}\}$ be the set of all possible functions with domain $\mathcal{X}$ and range $\mathcal{Y}$. Finally, for any key $k \xleftarrow{\$} \mathcal{K}$, let f_k denote the function $F(k, \cdot)$, i.e., $f_k(x) = F(k, x)$ for any $x \in \mathcal{X}$.

Definition 1 (PRF). A keyed function family $F : \mathcal{K} \times \mathcal{X} \rightarrow \mathcal{Y}$ as above is said to be a PRF family if for any uniformly randomly sampled $k \xleftarrow{\$} \mathcal{K}$, any uniformly randomly sampled function $g \xleftarrow{\$} \mathcal{F}$, and any probabilistic polynomial time (PPT) adversary $\mathcal{A}$, we have

$$\left| \Pr[\mathcal{A}^{f_k(\cdot)}(1^\lambda) = 1] - \Pr[\mathcal{A}^{g(\cdot)}(1^\lambda) = 1] \right| < \text{negl}(\lambda),$$

Weak PRF. We also recall the notion of a *weak* pseudorandom function (PRF). Informally speaking, a weak PRF is only guaranteed to be pseudorandom on uniformly random inputs (as opposed to arbitrary inputs in Definition 1). The formal definition is as follows.

Definition 2 (Weak PRF). A keyed function family $F : \mathcal{K} \times \mathcal{X} \rightarrow \mathcal{Y}$ as above is said to be a weak PRF family if for any uniformly randomly sampled $k \xleftarrow{\$} \mathcal{K}$, any $Q = \text{poly}(\lambda)$, any $x_1, \dots, x_Q \xleftarrow{\$} \mathcal{X}$, and any probabilistic polynomial time (PPT) adversary $\mathcal{A}$, we have

$$\left| \Pr[\mathcal{A}(1^\lambda, x_1, \dots, x_Q, f_k(x_1), \dots, f_k(x_Q)) = 1] - \Pr[\mathcal{A}(1^\lambda, x_1, \dots, x_Q, y_1, \dots, y_Q) = 1] \right| < \text{negl}(\lambda),$$

where, for each $i \in [Q]$, $y_i \xleftarrow{\$} \mathcal{Y}$.

2.2 Distributed PRF (DPRF)

We now recall the formal definition of a distributed PRF (DPRF) from [NPR99] below.

Definition 3 (Distributed Pseudo Random Function (DPRF)). Let $\mathcal{P} = \{P_j : j \in [T]\}$ be a set of T parties. A (t, T) -distributed PRF DPRF with input space $\mathcal{X}$, key space $\mathcal{K}$, and output space $\mathcal{Y}$ consists of three PPT algorithms as described below.

$$\text{DPRF} = (\text{DPRF.Setup}, \text{DPRF.PartialEval}, \text{DPRF.FinalEval}).$$

- DPRF.Setup($1^\lambda, t, T$): On input the security parameter λ , total number of parties T , and threshold number of parties t , this algorithm generates a key $k \xleftarrow{\$} \mathcal{K}$, and then shares it among T parties using some t -out-of- T threshold secret sharing scheme. At the end of the secret sharing process, the actual key k is not stored anywhere. Each party has to store one or more than one key share(s) depending on the particular threshold secret sharing scheme used.
- DPRF.PartialEval($x, P_j, \mathcal{P}'$): On input a valid subset $\mathcal{P}' \subset \mathcal{P}$ with $|\mathcal{P}'| = t$, an input $x \in \mathcal{X}$ and a party $P_j \in \mathcal{P}'$, the appropriate key share (say, k_j) of P_j corresponding to $\mathcal{P}'$ is chosen and a partial evaluation $f_{k_j}(x)$ is returned. Note that, if $|\mathcal{P}'| \geq t$, we consider any t parties among them to get a minimal valid subset and proceed in the same manner.
- DPRF.FinalEval($\mathcal{P}', \{f_{k_j}(x)\}_{P_j \in \mathcal{P}'}$): On input a valid subset $\mathcal{P}' \subset \mathcal{P}$ with $|\mathcal{P}'| = t$, and all the partial evaluations by parties $P_j \in \mathcal{P}'$, this algorithm combines them to get the final PRF evaluation. The actual combination procedure depends upon the reconstruction property of the underlying threshold secret sharing scheme.

Correctness and Consistency. A (t, T) -distributed PRF with $f_k(\cdot)$ as its underlying PRF is *correct* if given an input, its distributed evaluation by any valid subset $\mathcal{P}' \subset \mathcal{P}$ outputs the same value as would be obtained by directly evaluating $f_k(\cdot)$ on the same input except with negligible probability, i.e.,

$$\Pr[\text{DPRF.FinalEval}(\mathcal{P}', \{\text{DPRF.PartialEval}(x, P_j, \mathcal{P}')\}_{P_j \in \mathcal{P}'}) = f_k(x)] \geq 1 - \text{negl}(\lambda).$$

A (t, T) -distributed PRF is *consistent* if distributed evaluation on a given input by any two distinct valid subsets $S_1, S_2 \subset \mathcal{P}$ with $|S_1| = |S_2| = t$ outputs the same value except with negligible probability, i.e.,

$$\begin{aligned} & \left| \Pr[\text{DPRF.FinalEval}(S_1, \{\text{DPRF.PartialEval}(x, P_j, S_1)\}_{P_j \in S_1}) \right. \\ & \quad \left. \neq \text{DPRF.FinalEval}(S_2, \{\text{DPRF.PartialEval}(x, P_{j'}, S_2)\}_{P_{j'} \in S_2})] \right| \leq \text{negl}(\lambda). \end{aligned}$$

Note that the correctness of (t, T) -distributed PRF implies its consistency, but not the other way around.

Adaptive Security. We now recall the notion of adaptive security [LST21, Muk20] for a (t, T) -DPRF for any $t < T$ against a semi-honest PPT adversary that adaptively corrupts at most $(t - 1)$ out of T parties.

Let $\lambda \in \mathbb{N}$ be the security parameter. Let $\mathcal{P} = \{P_j : j \in [T]\}$ be the set of T parties. Consider the following game between a challenger Ch and a PPT adversary Ad:

- *Setup:* The challenger first chooses a secret $\mathbf{k} \in \mathcal{K}$. Then, $\mathbf{k}$ is distributed among the T parties $\{P_j : j \in [T]\}$ using some (t, T) -threshold secret sharing. The set of corrupted secret shares $\mathcal{L}$ is initially Φ (i.e., empty).
- *Partial Evaluation Query:* Ad adaptively queries Ch with $(x, P_j, \mathcal{P}')$, to see partial evaluations by party P_j on input x of its choice w.r.t a t -sized group $\mathcal{P}'$, such that $P_j \in \mathcal{P}'$, and $(\mathcal{P}', P_j) \notin \mathcal{L}$. In response, Ch computes the partial evaluation on input x using the secret share of P_j corresponding to t -sized group $\mathcal{P}'$.
- *Corruption Query:* Here, Ad sends corruption query of the form $(\mathcal{P}', P_j)$ to Ch such that $P_j \in \mathcal{P}'$ and $|\mathcal{P}'| = t$. In response, Ch sends the secret share of P_j corresponding to $\mathcal{P}'$ to Ad. The list $\mathcal{L}$ is appended with $(\mathcal{P}', P_j)$. For the same t -sized group $\mathcal{P}'$, the total number of corrupted parties in $\mathcal{L}$ should not exceed $(t - 1)$; otherwise, the game is aborted.
- *Challenge Phase:* In this phase, Ad provides Ch with a challenge input x^* . Now, Ch chooses a random bit $b \xleftarrow{\$} \{0, 1\}$. If $b = 0$, it sends the DPRF evaluation by any t parties of $\mathcal{P}$ on challenge input x^* to Ad. Otherwise, it sends $y^* \xleftarrow{\$} \mathcal{Y}$, i.e., a truly random output over the range of the PRF, to Ad.
- *Post-challenge Partial Evaluation Query:* This is exactly same as the pre-challenge partial evaluation query phase; however, Ad is allowed to raise partial evaluation query on challenge input x^* with a limitation that for same t -sized subset $\mathcal{P}'$, total number of distinct pre-challenge and post-challenge partial evaluation queries on x^* should not exceed $(t - 1)$. Otherwise, the adversary can itself learn t -many partial evaluations of challenge input x^* , compute the DPRF evaluation on x^* itself and distinguish it from a uniform random value.
- *Post-challenge Corruption Query:* This phase is exactly the same as the pre-challenge corruption query phase.
- *Output Phase:* Finally, Ad outputs a distinguishing bit b' .

Note that partial evaluation queries (upper-bounded by polynomial in λ) and corruption queries can be raised in any order throughout the game. The challenge phase occurs only once and can happen anytime during the game.

The DPRF is said to be adaptively secure if for any $\lambda \in \mathbb{N}$ and any PPT adversary Ad that is allowed adaptive corruptions in the above game, we have

$$\Pr[b' = 1 | b = 0] - \Pr[b' = 1 | b = 1] \leq \text{negl}(\lambda).$$

We finally remark that static security for a DPRF is defined in exactly the same way as above, except that the adversary Ad is not allowed to issue corruption queries adaptively (in other words, all corruption queries must be issued by Ad statically at setup).

Separation between Static and Adaptive Security. The following theorem establishes a *separation* between static and adaptive security of DPRFs (see [LST21] for a formal proof).

Theorem 1 (Adapted from Theorem 2.24 from [LST21]). *For any $t, T \in \text{poly}(\lambda)$, such that $\binom{T}{t-1}$ is super-polynomial, there exists a family of distributed pseudorandom functions, which is secure in the static corruption model but insecure in the adaptive corruption model.*

In particular, the above theorem establishes that, for any $t, T \in \text{poly}(\lambda)$ such that $t < T$ and such that $\binom{T}{t-1}$ is super-polynomial, it is non-trivial to achieve an adaptively secure (t, T) -DPRF even given a statically secure (t, T) -DPRF. In this paper, we focus on achieving a (t, T) -DPRF for precisely such choices of (t, T) based on the module LWR assumption in the QROM.

2.3 Lattice-based Computational Assumptions

In this subsection, we recall the Learning with Rounding (LWR) assumption and its module variant (called Module LWR), which forms the basis of our core DPRF construction.

Learning with Rounding (LWR) [BPR12]. An LWR sample for parameters (n, q, p) is a tuple of the form $(\mathbf{a}, b = \lfloor \langle \mathbf{a}, \mathbf{k} \rangle \rfloor_p)$ where:

- $\mathbf{a} = (a_1, \dots, a_n), \mathbf{k} = (k_1, \dots, k_n) \xleftarrow{\$} \mathbb{Z}_q^n$ are both vectors with entries sampled uniformly at random.
- $\langle \mathbf{a}, \mathbf{k} \rangle$ denotes the inner product $\sum_{i=1}^n a_i \cdot k_i$.

The (n, q, p) -LWR assumption posits that the following ensembles are computationally indistinguishable:

$$(\mathbf{a}, b = \lfloor \langle \mathbf{a}, \mathbf{k} \rangle \rfloor_p) \approx_c (\mathbf{a}, u)$$

where $(\mathbf{a}, b)$ is an MLWR sample as described above for uniformly randomly sampled $\mathbf{a}, \mathbf{k} \xleftarrow{\$} \mathbb{Z}_q^n$, and where $u \xleftarrow{\$} \mathbb{Z}_p^n$ is uniformly randomly sampled. In other words, the (n, q, p) -LWR assumption posits that no PPT adversary can distinguish between the above ensembles. LWR is essentially a “derandomized” analogue of Learning with Errors (LWE) [Reg09].

Module Learning with Rounding (MLWR). A Module Learning with Rounding (MLWR) sample for parameters (N, d, q, p) is a tuple of the form $(\mathbf{A}, B = \lfloor \mathbf{A} \cdot \mathbf{K} \rfloor_p)$, where:

- $\mathbf{A} = (\mathbf{A}_1, \dots, \mathbf{A}_d) \xleftarrow{\$} \mathcal{R}_{N,q}^d$ and $\mathbf{K} = (\mathbf{K}_1, \dots, \mathbf{K}_d) \xleftarrow{\$} \mathcal{R}_{N,q}^d$ are both sampled uniformly at random.
- $\mathbf{A} \cdot \mathbf{K}$ denotes the inner product $\sum_{i=1}^d \mathbf{A}_i \cdot \mathbf{K}_i$, where, for each $i \in [d]$, $\mathbf{A}_i \cdot \mathbf{K}_i$ denotes the polynomial multiplication of $\mathbf{A}_i$ and $\mathbf{K}_i$ modulo $(x^N + 1)$.

The (N, d, q, p) -MLWR assumption posits that the following ensembles are computationally indistinguishable :

$$(\mathbf{A}, B = \lfloor \mathbf{A} \cdot \mathbf{K} \rfloor_p) \approx_c (\mathbf{A}, U)$$

where $(\mathbf{A}, B)$ is an MLWR sample as described above, and where $U \xleftarrow{\$} \mathcal{R}_{N,p}$ is uniformly randomly sampled. In other words, the (N, d, q, p) -MLWR assumption posits that no probabilistic polynomial-time (PPT) adversary can distinguish between the above ensembles. Again, module LWR is essentially a “derandomized” analogue of module LWE [BGV14, LS15], which in turn is a variant of LWE with respect to *structured* (module) lattices.

2.4 Some Basic Lemmata

In this section, we recall the leftover hash lemma [ILL89, Reg09] and its extension to modules [LPR13], which we use to prove the security of our DPRF construction.

Lemma 1 (Leftover Hash Lemma [Reg09]). *Let X be a random variable over a finite set $\mathcal{X}$ with min-entropy $H_\infty(X) \geq k$, and let $\tilde{\mathcal{H}}$ be a universal family of hash functions mapping $\mathcal{X}$ to a range $\mathcal{Y}$ of size M . Then, for a uniformly sampled $\mathcal{H} \xleftarrow{\$} \tilde{\mathcal{H}}$, the joint distribution $(\mathcal{H}, \mathcal{H}(X))$ is at statistical distance at most ϵ from $(\mathcal{H}, \mathcal{U}(\mathcal{Y}))$ whenever,*

$$k \geq \log M + 2 \log \frac{1}{\epsilon}.$$

In particular, for a uniformly random matrix $\mathbf{A} \in \mathbb{Z}_q^{m \times n}$, the family of linear maps

$$\mathcal{H}_{\mathbf{A}}(\mathbf{s}) = \mathbf{A} \cdot \mathbf{s} \pmod{q},$$

is universal. Consequently, if $\mathbf{s} \xleftarrow{\$} \{0, 1\}^n$, then for $n \geq m \log_2 q + 2 \log_2 \frac{1}{\epsilon}$, the following holds:

$$\Delta((\mathbf{A}, \mathbf{A} \cdot \mathbf{s}), (\mathbf{A}, \mathcal{U}(\mathbb{Z}_q^m))) \leq \epsilon.$$

We now recall an extension of the above lemma to modules [LPR13].

Lemma 2 (Regularity Lemma over Modules (Adapted from Theorem 4.1 and Corollary 4.2 of [LPR13])). *Let $\mathbf{A} = [\mathbf{I}_d | \tilde{\mathbf{A}}] \leftarrow \mathcal{R}_{N,q}^{d \times \ell}$, such that $\tilde{\mathbf{A}} \xleftarrow{\$} \mathcal{R}_{N,q}^{d \times (\ell-d)}$ is uniformly random and $\mathbf{I}_d$ is the identity matrix. Then with probability $1 - 2^{-\Omega(N)}$ over the choice of $\tilde{\mathbf{A}}$, the distribution of $\mathbf{A} \cdot \mathbf{s} \in \mathcal{R}_{N,q}^d$, where each element of $\mathbf{s} \in \mathcal{R}_{N,q}^\ell$ is sampled from a discrete Gaussian distribution of radius $r > 2N \cdot q^{\frac{d}{\ell} + \frac{2}{N\ell}}$ over $\mathcal{R}$, is within statistical distance $2^{-\Omega(N)}$ of the uniform distribution over $\mathcal{R}_{N,q}^d$.*

3 Adaptively Secure DPRF from MLWR

In this section, we propose a (t, T) -distributed PRF (DPRF) that achieves adaptive security for any $t < T$ in the QROM based on the MLWR assumption. The rest of the section is organized as follows. In Section 3.1, we recall the non-adaptive, plain LWR-based (t, T) -DPRF construction from [SPM24] (this is done for ease of exposition). In Section 3.2, we describe our MLWR-based DPRF construction, which essentially extends the LWR-based construction from [SPM24] to the module setting. In Section 3.3.3, we prove that the adaptive security of our construction in the QROM. Finally, Section 3.4 discusses concrete parameter choices.

3.1 Background: LWR-based Non-Adaptive DPRF [SPM24]

Let λ be a security parameter. Let $n = \text{poly}(\lambda)$. Let $q, q_1, p \in \mathbb{N}$ be suitably chosen $O(\lambda)$ -bit moduli such that $q > q_1 > p$. Finally, let $\mathcal{H}(\cdot) : \{0, 1\}^* \rightarrow \mathbb{Z}_q^n$ be a hash function (modeled as a random oracle). Let $\mathcal{P} = \{P_j : j \in [T]\}$ be a set of T parties. Without loss of generality, we assume $\mathcal{P}' = \{P_1, \dots, P_t\}$ to be a t -sized subset of $\mathcal{P}$. The (t, T) -DPRF of [SPM24] with input space $\mathcal{X} = \{0, 1\}^*$, key space $\mathcal{K} = \mathbb{Z}_q^n$ and output space $\mathcal{Y} = \mathbb{Z}_p$ is as described below.

Construction 1 (LWR-based DPRF (Construction 2 in [SPM24])). *The construction is as follows:*

• Setup($1^\lambda, t, T$):

- Sample the master secret key $\mathbf{k} \xleftarrow{\$} \mathcal{K}$, and share it among T parties using Benaloh-Leichter linear integer secret sharing scheme [DT06].
- After the secret sharing process, each $P_j \in \mathcal{P}$ receives a total of $\binom{T-1}{t-1}$ shares each corresponding to one of the t -size subsets of $\mathcal{P}$, that P_j belong to. Each of the shares is an element of $\mathbb{Z}_q^n$.
- At the end of the secret sharing phase, the master secret key $\mathbf{k}$ is destroyed.

- PartialEvaluation($x, P_j, \mathcal{P}'$):
 - Assume $\mathcal{P}'$ is a minimal valid subset containing P_j , i.e., $P_j \in \mathcal{P}'$ and $|\mathcal{P}'| = t$. Also, assume $\mathbf{k}_j$ to be the secret share of P_j corresponding to subset $\mathcal{P}'$.
 - Given input $\mathbf{x}$, P_j computes the partial evaluation as $\mu_j = \lfloor \langle \mathcal{H}(\mathbf{x}), \mathbf{k}_j \rangle \rfloor_{q_1} \in \mathbb{Z}_{q_1}$, and sends it to a designated evaluator in $\mathcal{P}'$.
- FinalEvaluation($\{\mu_j\}_{P_j \in \mathcal{P}'}, \mathcal{P}'$): In presence of all t partial evaluations from t parties in $\mathcal{P}'$, the final evaluation leverages the reconstruction property of the linear integer secret sharing scheme [DT06] (i.e., $\mathbf{k}_1 - \sum_{j=2}^t \mathbf{k}_j = \mathbf{k}$), and computes the following:

$$f_{\{\mathbf{k}_j\}_{j \in [t]}}^{\text{dist}}(\mathbf{x}) = \lfloor \mu_1 - \sum_{j=2}^t \mu_j \rfloor_p$$

We note the following observations about the above construction:

- The authors of [SPM24] prove the *non-adaptive* security of the above DPRF construction in the ROM based on the LWR assumption. In particular, their proof strategy relies crucially on knowing the entire set of corrupted parties before the adversary is allowed to query for any partial evaluations on inputs of its choice. Their proof *does not* naturally extend to the adaptive setting, where the adversary is allowed to corrupt parties and receive the corresponding key shares on the fly.
- Informally speaking, the authors of [SPM24] propose choosing parameters (n, p, p_1, q_1) as follows to ensure both correctness and security: (i) the ratios $\frac{p}{q_1}$ and $\frac{q_1}{q}$ should be sufficiently small (to preserve correctness), and (ii) n should be sufficiently large for the (n, q, p) -LWR assumption to hold (which in turn guarantees security). The choice of parameters is justified via the LWE estimator [APS15].

3.2 Our Module-LWR based DPRF

We now present our main contribution, namely a (t, T) -DPRF that achieves adaptive security for any $t < T$ in the QROM based on the MLWR assumption. Again, let λ be a security parameter. Let $N, d = \text{poly}(\lambda)$. Let $q, q_1, p \in \mathbb{N}$ be suitably chosen $O(\lambda)$ -bit moduli such that $q > q_1 > p$. Finally, let $\mathcal{H}(\cdot) : \{0, 1\}^* \rightarrow \mathcal{R}_{N, q}^d$ be a hash function (modeled subsequently in the security proof as a quantum-accessible random oracle). Let $\mathcal{P} = \{P_j : j \in [T]\}$ be a set of T parties. Without loss of generality, we assume $\mathcal{P}' = \{P_1, \dots, P_t\}$ to be a t -sized subset of $\mathcal{P}$. Construction 2 describes our module LWR-based DPRF

$$\text{DPRF} = (\text{DPRF.Setup}, \text{DPRF.PartialEval}, \text{DPRF.FinalEval})$$

with input space $\mathcal{X} = \{0, 1\}^*$, key space $\mathcal{K} = \mathcal{R}_{N, q}^d$ and output space $\mathcal{Y} = \mathcal{R}_{N, p}$.

Construction 2 ((t, T) -DPRF from MLWR). *The construction is as follows:*

- DPRF.Setup($1^\lambda, t, T$):
 - Sample the master secret key $\mathbf{K} \xleftarrow{\$} \mathcal{K}$, and share $\mathbf{K}$ among $\mathcal{P} = \{P_1, \dots, P_T\}$ using the linear integer secret sharing (LISS) scheme of [DT06]. The LISS scheme interprets the secret $\mathbf{K}$ as a vector of $d \cdot N$ elements in $\mathbb{Z}_q$.
 - At the end of the secret sharing procedure, each party $P_j \in \mathcal{P}$ for $j \in [T]$ receives a secret key share $\tilde{\mathbf{K}}_j$ – a vector of $\binom{T-1}{t-1}$ shares, each corresponding to one of the t -sized subsets (of $\mathcal{P}$) that P_j belongs to. Each share in $\tilde{\mathbf{K}}_j$ is, in turn, a vector of $d \cdot N$ elements of $\mathbb{Z}_q$.
 - Post-sharing, the master secret key $\mathbf{K}$ is destroyed.

- DPRF.PartialEval($x, P_j, \mathcal{P}'$):

- Assume $\mathbf{K}_j$ is the secret share of P_j corresponding to a t -sized subset $\mathcal{P}'$ such that $P_j \in \mathcal{P}'$.
- Given an input x , partial evaluation on x by P_j corresponding to $\mathcal{P}'$ is computed using $\mathbf{K}_j$ as $\mu_j = f_{\mathbf{K}_j}(x) = \lfloor \mathcal{H}(x) \cdot \mathbf{K}_j \rfloor_{q_1}$.

- DPRF.FinalEval($\{\mu_j\}_{P_j \in \mathcal{P}'}, \mathcal{P}'$): Given t partial evaluations from parties in $\mathcal{P}'$, the final evaluation is computed as:

$$f_{\{\mathbf{K}_j\}_{P_j \in \mathcal{P}'}}^{\text{dist}}(x) = \lfloor \mu_1 - \sum_{j=2}^t \mu_j \rfloor_p.$$

Correctness of Construction 2. Now, we argue the correctness of the proposed MLWR-based DPRF construction in the following.

Theorem 2. *The proposed DPRF in Construction 2, with proper choice of parameters, satisfies correctness and consistency as described in Section 2.2.*

Proof. We require $f_{\mathbf{K}}^{\text{base}}(x)$ and $f_{\{\mathbf{K}_j\}_{P_j \in \mathcal{P}'}}^{\text{dist}}(x)$ to be the same for any choice of $\mathcal{P}'$ to argue the correctness of the proposed DPRF. Using following two facts,

- for any $A \in \mathcal{R}_{N,q}$, $\lfloor A \rfloor_p$ can be expressed as $A \cdot \frac{p}{q} + E$, where E is an N -sized error polynomial with each coefficient in $[\frac{1}{2}, \frac{1}{2})$, and,
- for any minimal valid subset $\mathcal{P}'$ of size t with key shares $\{\mathbf{K}_j\}_{j \in [t]}$, $\mathbf{K} = \mathbf{K}_1 - \sum_{i=2}^t \mathbf{K}_i$ holds,

one can follow simple derivation steps to reach the following expressions:

$$f_{\mathbf{K}}^{\text{base}}(x) = \frac{p}{q} \cdot (\mathcal{H}(x) \cdot \mathbf{K}) + E', \quad f_{\{\mathbf{K}_j\}_{P_j \in \mathcal{P}'}}^{\text{dist}}(x) = \frac{p}{q} \cdot (\mathcal{H}(x) \cdot \mathbf{K}) + \frac{p}{q_1} \cdot (E_1 - \sum_{j=2}^t E_j) + \hat{E},$$

where $E', \{E_j\}_{j \in [t]}, \hat{E}$ are distinct error polynomials with coefficients in $[\frac{1}{2}, \frac{1}{2})$. Clearly,

$$|f_{\{\mathbf{K}_j\}_{P_j \in \mathcal{P}'}}^{\text{dist}}(x) - f_{\mathbf{K}}^{\text{base}}(x)| \leq \frac{p}{q_1} \cdot \left| \sum_{j=1}^t E_j \right| + |\hat{E} - E'|.$$

Note that each coefficient of $|\hat{E} - E'|$ is less than 1 and each coefficient of $\frac{p}{q_1} \cdot \left| \sum_{j=1}^t E_j \right|$ is less than $\frac{pt}{2q_1}$. Hence, with super-polynomial $\frac{q_1}{p}$ ratio, DPRF is correct with overwhelming probability.

However, with practical choice of q_1, p , with polynomial $\frac{q_1}{p}$, we obtain small correctness error, denoted with ϵ_{corr} . While for many applications, a small (but non-negligible) correctness error may be acceptable, for applications demanding higher accuracy, we achieve negligible correctness error by repeating the evaluation parallelly $\text{rep} = \lceil \frac{\lambda}{-\log \epsilon_{\text{corr}}} \rceil$ times for rep different sets of secret shares¹ and accepting the output only if all these outputs match. If all values do not match, we re-evaluate the DPRF in the same manner using some other valid subset. $\square$

Size of Secret Shares. Each party holds $\binom{T-1}{t-1}$ shares, each belonging to $\mathcal{R}_q^d$, thus requiring $(\binom{T-1}{t-1} \cdot N \cdot d \cdot \log q)$ bits of storage. When we consider repetition factor rep for negligible correctness error, the storage requirement is increased by a constant multiplication factor of rep , which is $(\binom{T-1}{t-1} \cdot N \cdot d \cdot \log q \cdot \text{rep})$ bits. In summary, the asymptotic storage complexity is $O(\binom{T-1}{t-1})$.

¹This technique applies to [SPM24] also to achieve negligible correctness error.

3.3 Adaptive Security of MLWR-based DPRF

In this section, we prove the adaptive security of the above (t, T) -DPRF for any $t < T$ in the QROM based on the MLWR assumption. The proof is broken into three parts for ease of exposition:

- As a warm-up, we first prove in Section 3.3.1 that the LWR-based DPRF of [SPM24] (Construction 1 in Section 3.1) is, in fact, adaptively secure in the (classical) ROM (based on the same assumption, albeit for slightly different choice of parameters). This is formally captured in Theorem 3.
- As a next step, we prove in Section 3.3.2 Construction 1 remains adaptively secure even when the hash function is modeled as a quantum-accessible ROM (QROM). This is formally captured in Theorem 4.
- Finally, we extend the above proof of adaptive security to the module setting in Section 3.3.3, thus establishing the adaptive security of Construction 2 in the QROM, as desired. This is formally captured in Theorem 5, which is our main security theorem.

3.3.1 Adaptive Security of LWR-based DPRF [SPM24] in the ROM

We state and prove the following theorem.

Theorem 3 (Adaptive Security of LWR-based DPRF in the ROM). *Assuming the hardness of LWR, Construction 1 is adaptively secure in the random oracle model (ROM).*

Proof. First, we prove the adaptive security of Construction 1 as described in Section 3.1 by showing indistinguishability through a chain of hybrids in ROM. Each hybrid has an initial setup phase and then describes a set of interactions between a challenger Ch and an adversary Ad. The interaction can be broadly categorized into (i) the query phase and (ii) the challenge phase. The query phase (consisting of random oracle queries, partial evaluation queries, and corruption queries in any order) may occur before and after the challenge phase. There can be polynomially many random oracle queries and partial evaluation queries. Challenge phase consists of a single query from Ad with an input of its choice (called the challenge input), and a single response from the Ch which can either be the DPRF evaluation on challenge input or a uniform random value from the DPRF range. Ad is allowed to choose the challenge input arbitrarily, (i.e., the challenge input can be any one of the pre-challenge query inputs), however Ch does not know beforehand which query will correspond to challenge query. Let $Q = \text{poly}(\lambda)$ be the total number of partial evaluation queries sent by Ad to Ch. At the beginning of the game, the challenger guesses a value $c \xleftarrow{\$} [Q + 1]$, and considers c^{th} query input to be the challenge input. The game aborts if the guess is wrong. The random guess is correct with non-negligible probability $\frac{1}{Q+1}$, thus, incurring a polynomial loss in the reduction. Also, when $c = Q + 1$, it implies that there is no post-challenge partial evaluation query.

Recall that a negligible correctness error is achieved by repeating the complete DPRF evaluation rep times independently for rep different sets of shares for the same base PRF key $\mathbf{k}$, and accepting the DPRF output only if it is the same for all rep cases. Next, we describe the hybrids in detail.

Hybrid₀. This is the real game between Ch and Ad. In the challenge phase, Ad gets to see the result of distributed PRF evaluation on the challenge input.

Setup: Let $\mathcal{P} = \{P_i : i \in [T]\}$ be the set of T parties. Ch chooses a uniformly random PRF secret $\mathbf{k} \xleftarrow{\$} \mathbb{Z}_q^n$, and distributes it among T parties using a (t, T) -threshold secret sharing scheme [DT06], rep -many times. For each of rep independent sets of shares, each party $P_i \in \mathcal{P}$ receives $\binom{T-1}{t-1}$ secret shares, corresponding to the t -sized subsets of $\mathcal{P}$ that include P_i . Ch maintains a list of compromised secret shares $\mathcal{L}$, which is initially empty. When Ad raises a corruption query asking for the share of P_j with respect to the subset $\mathcal{P}'$ (such that $P_j \in \mathcal{P}'$ and $|\mathcal{P}'| = t$), $\mathcal{L}$ is appended with the entry $(\mathcal{P}', P_j)$. For any fixed subset $\mathcal{P}'$, $\mathcal{L}$ must contain fewer than t entries corresponding to different P_j 's. If $\mathcal{L}$ accumulates t or more entries with the same $\mathcal{P}'$ but different P_j 's, the secret $\mathbf{k}$ can be reconstructed, allowing the adversary to win the game trivially. Hence, Ch aborts the game on violation of this condition.

Random oracle query: When a query with an input $\mathbf{x}_i$ is received for the first time, the oracle samples a uniform random element from $\mathbb{Z}_q^n$, denoted as $\mathcal{H}(\mathbf{x}_i) \xleftarrow{\$} \mathbb{Z}_q^n$, and stores the pair $(\mathbf{x}_i, \mathcal{H}(\mathbf{x}_i))$, following the lazy sampling strategy in ROM.

Partial evaluation query: Ad submits a query of the form $(\mathbf{x}_i, P_j, \mathcal{P}')$ asking for the partial evaluation on input $\mathbf{x}_i$ for party P_j and t -sized subset $\mathcal{P}'$, such that $P_j \in \mathcal{P}'$ and $(\mathcal{P}', P_j) \notin \mathcal{L}$. Let $\mathbf{k}_{j,r} \forall r \in [\text{rep}]$ be the secret shares of P_j corresponding to $\mathcal{P}'$, for rep sets of shares. Ch responds with

$$[\langle \mathcal{H}(\mathbf{x}_i), \mathbf{k}_{j,1} \rangle]_{q_1}, \dots, [\langle \mathcal{H}(\mathbf{x}_i), \mathbf{k}_{j,r} \rangle]_{q_1}, \dots, [\langle \mathcal{H}(\mathbf{x}_i), \mathbf{k}_{j,\text{rep}} \rangle]_{q_1} \in \mathbb{Z}_{q_1}^{\text{rep}},$$

an array of rep partial evaluations, each corresponding to one of the rep sets of shares.

Corruption query: Ad raises query of the form $(\mathcal{P}', P_j)$ to Ch such that $P_j \in \mathcal{P}'$, $|\mathcal{P}'| = t$, and $(\mathcal{P}', P_j) \notin \mathcal{L}$. Ch responds with $[\mathbf{k}_{j,1}, \dots, \mathbf{k}_{j,r}, \dots, \mathbf{k}_{j,\text{rep}}] \in \mathbb{Z}_q^{n \times \text{rep}}$, i.e., the secret shares of $(\mathcal{P}', P_j)$ corresponding to rep different sets of shares. The corruption list gets updated as $\mathcal{L} = \mathcal{L} \cup \{(\mathcal{P}', P_j)\}$.

Challenge phase: Ad submits a challenge input $\mathbf{x}^*$ to Ch. Now, Ch computes rep DPRF evaluations on $\mathbf{x}^*$ as $f_{\{\mathbf{k}_{j,r}\}_{P_j \in \mathcal{P}'}}^{\text{dist}}(\mathbf{x}^*)$ for all $r \in [\text{rep}]$ and checks if all of them are equal. If they are, Ch returns this result to Ad. By the correctness of the DPRF, this value equals $[\langle \mathcal{H}(\mathbf{x}^*), \mathbf{k} \rangle]_p$. Here, $\mathcal{P}'$ can be any t -sized subset of $\mathcal{P}$. If any one of the rep evaluations does not match with others, Ch uses some other valid subset to have rep independent evaluations and checks their equality. And, with overwhelming probability the rep evaluations turn out to be equal this time.

Hybrid₁. In this hybrid, Ch responds differently only while answering random oracle queries. For other queries like the partial evaluation query and the corruption query, its behavior remains exactly the same as in Hybrid₀. In the challenge phase also, Ch responds in the same manner as Hybrid₀.

Setup: Two matrices $\mathbf{A} \xleftarrow{\$} \mathbb{Z}_q^{n \times m}$ and $\mathbf{R} \xleftarrow{\$} \mathbb{Z}_q^{n \times n}$ are chosen such that $\mathbf{R}$ is invertible modulo q . If $\mathbf{R}$ is not invertible, then it goes on sampling new $\mathbf{R}$ until it is invertible¹.

Random oracle query: Random oracle is programmed as follows: for every query from Ad with a new input $\mathbf{x}_i$, a short-norm vector $\mathbf{s}_i \in \{0, 1\}^m$ is sampled and $\mathcal{H}(\mathbf{x}_i) = (\mathbf{R} \cdot \mathbf{A}) \cdot \mathbf{s}_i \in \mathbb{Z}_q^{n \times m}$ is returned. For each queried input $\mathbf{x}_i$, $(\mathbf{s}_i, \mathcal{H}(\mathbf{x}_i))$ are stored, following the lazy sampling strategy.

Lemma 3. Ad can distinguish Hybrid₁ from Hybrid₀ with advantage $\epsilon_{0,1}$ for some $\epsilon_{0,1} = \text{negl}(\lambda)$.

Proof. Since the two hybrids differ only in the random oracle query phase, we prove that they can not be distinguished from each other from Ch's responses to random oracle queries. Note that for fixed and invertible matrix $\mathbf{R} \in \mathbb{Z}_q^{n \times n}$, and uniform random $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$, $\mathbf{R} \cdot \mathbf{A}$ is also uniform random over $\mathbb{Z}_q^{n \times m}$. Now, the responses to random oracle queries in Hybrid₁, i.e., $(\mathbf{R} \cdot \mathbf{A}) \cdot \mathbf{s}_i$ for any input $\mathbf{x}_i$ is statistically indistinguishable from uniform random over $\mathbb{Z}_q^n$, due to leftover hash lemma (Lemma 1) for sufficiently large m . Thus, the random oracle query phases of Hybrid₀ and Hybrid₁ are statistically indistinguishable. We conclude that Ad has distinguishing advantage of $\epsilon_{0,1}$ for some $\epsilon_{0,1} = \text{negl}(\lambda)$, while transitioning from Hybrid₀ to Hybrid₁. $\square$

Hybrid₂. In this hybrid, only the random oracle query phase differs slightly from Hybrid₁. Interactions between Ch and Ad in all other phases are same as Hybrid₁.

Random oracle query: For any non-challenge input $\mathbf{x}_i$, the response $\mathcal{H}(\mathbf{x}_i)$ is still $(\mathbf{R} \cdot \mathbf{A}) \cdot \mathbf{s}_i$ for some short-norm vector $\mathbf{s}_i$. However, the oracle is programmed differently at the challenge input $\mathbf{x}^*$ - a short-norm vector $\mathbf{s}^* \in \{0, 1\}^m$ is sampled and $\mathcal{H}(\mathbf{x}^*) = \mathbf{A} \cdot \mathbf{s}^* \in \mathbb{Z}_q^n$ is returned.

Lemma 4. Ad can distinguish Hybrid₂ from Hybrid₁ with an advantage of $\epsilon_{1,2}$ for some $\epsilon_{1,2} = \text{negl}(\lambda)$.

¹ $\mathbf{R}$ is invertible if it is full-rank and its determinant is co-prime to q . The first event has an approximate probability of 0.28. The second event is independent of the first one and has a probability of 0.5 since we choose q to be a power of 2. Thus $\mathbf{R}$ is invertible with an approximate non-negligible probability of 0.14.

Proof. Since the value of $\mathcal{H}(\mathbf{x}^*)$ is statistically indistinguishable from uniform random over $\mathbb{Z}_q^n$ in both the hybrids, due to leftover hash lemma (Lemma 1) for sufficiently large m , we conclude that Ad has distinguishing advantage of $\epsilon_{1,2}$ for some $\epsilon_{1,2} = \text{negl}(\lambda)$, while transitioning from Hybrid₁ to Hybrid₂. $\square$

Hybrid_{3,1}. In this hybrid, the random oracle query phase is same as Hybrid₂. But the responses to partial evaluation queries and corruption queries are partially simulated - i.e., the responses corresponding to the first set of secret shares are simulated without using the real shares $\mathbf{k}_{j,1}$, but for all other sets where $r \in [\text{rep}] \setminus \{1\}$, Ch's responses are the same as Hybrid₂.

Partial evaluation query: Broadly, Ch's response to a partial evaluation query of the form $(\mathbf{x}_i, P_j, \mathcal{P}')$ is in $\mathbb{Z}_{q_1}^{\text{rep}}$, where the first element in the vector is simulated, and the last $(\text{rep} - 1)$ are real. For all $r \in [2, \text{rep}]$, Ch responds with $\lfloor \langle \mathcal{H}(\mathbf{x}_i), \mathbf{k}_{j,r} \rangle \rfloor_{q_1}$ in case of any non-challenge input $\mathbf{x}_i$, and with $\lfloor \langle \mathcal{H}(\mathbf{x}^*), \mathbf{k}_{j,r} \rangle \rfloor_{q_1}$ in case of challenge input.

However, for $r = 1$, simulated responses vary for challenge and non-challenge inputs. Notably, Ch does not use the real shares $\mathbf{k}_{j,1}$ for any $(\mathcal{P}', P_j)$.

- *Non-challenge input:* On first query of the form $(\mathbf{x}_i, P_j, \mathcal{P}')$ for the pair $(\mathcal{P}', P_j)$ such that $P_j \in \mathcal{P}'$, $|\mathcal{P}'| = t$, and $(\mathcal{P}', P_j) \notin \mathcal{L}$, Ch samples a uniform random $\mathbf{k}'_{j,1} \xleftarrow{\$} \mathbb{Z}_q^n$, and responds to Ad with simulated partial evaluation $\lfloor \langle \mathbf{A} \cdot \mathbf{s}_i, \mathbf{k}'_{j,1} \rangle \rfloor_{q_1}$. Ch continues to use the same $\mathbf{k}'_{j,1}$ to answer queries with other inputs $\mathbf{x}_{i'}$'s for the same pair $(\mathcal{P}', P_j)$ and $r = 1$.
- *Challenge input:* If the partial evaluation query corresponding to $(\mathcal{P}', P_j)$ is raised for the first time and $\mathcal{L}$ contains less than $(t - 1)$ distinct entries for $\mathcal{P}'$, a uniform random $\mathbf{k}'_{j,1} \xleftarrow{\$} \mathbb{Z}_q^n$ is sampled for the pair. Then a simulated share is computed as $\mathbf{k}''_{j,1} = \mathbf{R}^{-1} \cdot \mathbf{k}'_{j,1}$. The partial evaluation queries on challenge input $\mathbf{x}^*$ are responded with $\lfloor \langle \mathcal{H}(\mathbf{x}^*), \mathbf{k}''_{j,1} \rangle \rfloor_{q_1}$. Otherwise, if $\mathbf{k}'_{j,1}$ already existed due to previous queries for the pair $(\mathcal{P}', P_j)$, $\mathbf{k}''_{j,1}$ is computed in the same manner, and $\lfloor \langle \mathcal{H}(\mathbf{x}^*), \mathbf{k}''_{j,1} \rangle \rfloor_{q_1} = \lfloor \langle \mathbf{A} \cdot \mathbf{s}^*, \mathbf{k}''_{j,1} \rangle \rfloor_{q_1}$ is returned to Ad. Further, if the partial evaluation query corresponding to $(\mathcal{P}', P_j)$ is raised for the first time but $\mathcal{L}$ contains exactly $(t - 1)$ distinct entries for $\mathcal{P}'$ (i.e., Ad has reached the corruption threshold), then the response must be consistent with final DPRF evaluation on $\mathbf{x}^*$ which is visible to Ad in the challenge phase. Hence, the partial evaluation on $\mathbf{x}^*$ in this case is simulated as $\lfloor v \cdot \frac{q}{q_1} - \sum_{(\mathcal{P}', P_j) \in \mathcal{L}} \langle \mathcal{H}(\mathbf{x}^*), \mathbf{k}''_{j,1} \rangle \rfloor_{q_1}$, where v is the response in the challenge phase. We provide the exact formulation of v in the description of challenge phase.

Note that when Ad is yet to achieve maximal corruption, then, for $r = 1$, the responses to partial evaluation queries use a random secret $\mathbf{k}'_{j,1}$ while simulating the partial evaluations of non-challenge inputs, but involve use of a simulated secret $\mathbf{k}''_{j,1}$ while responding to partial evaluation queries on challenge input. Informally, responses to partial evaluation queries on challenge input remain *consistent* in the view of the adversary with respect to simulated share $\mathbf{k}''_{j,1}$ which might get revealed eventually (or, have been revealed already) through corruption query phase. Responses to partial evaluation queries on non-challenge inputs do not require special handling when Ad reaches the corruption threshold, because actual DPRF evaluation on non-challenge inputs are never visible to Ad.

Corruption query: For $r \in [2, \text{rep}]$, Ch responds to corruption query of the form $(\mathcal{P}', P_j)$ with real shares $\mathbf{k}_{j,r}$, similar to Hybrid₂. However for $r = 1$, on corruption query of the form $(\mathcal{P}', P_j)$, the simulated share $\mathbf{k}''_{j,1}$ is computed as $\mathbf{R}^{-1} \cdot \mathbf{k}'_{j,1}$, and returned to Ad. At this point, it is important to check that the response to corruption query is consistent with the previous responses to partial evaluation queries.

For the non-challenge inputs with $r = 1$, in the view of the adversary, $\mathcal{H}(\mathbf{x}_i) = (\mathbf{R} \cdot \mathbf{A}) \cdot \mathbf{s}_i$, and the compromised share corresponding to the pair $(\mathcal{P}', P_j)$ is $\mathbf{k}''_{j,1}$. So, Ad checks if $\lfloor \langle \mathcal{H}(\mathbf{x}_i), \mathbf{k}''_{j,1} \rangle \rfloor_{q_1}$ matches with the previous partial evaluation query response $\lfloor \langle \mathbf{A} \cdot \mathbf{s}_i, \mathbf{k}'_{j,1} \rangle \rfloor_{q_1}$. The check passes since,

$$\lfloor \langle \mathcal{H}(\mathbf{x}_i), \mathbf{k}''_{j,1} \rangle \rfloor_{q_1} = \lfloor \langle (\mathbf{R} \cdot \mathbf{A}) \cdot \mathbf{s}_i, \mathbf{k}''_{j,1} \rangle \rfloor_{q_1} = \lfloor \langle \mathbf{A} \cdot \mathbf{s}_i, \mathbf{R} \cdot \mathbf{k}''_{j,1} \rangle \rfloor_{q_1} = \lfloor \langle \mathbf{A} \cdot \mathbf{s}_i, \mathbf{k}'_{j,1} \rangle \rfloor_{q_1}.$$

For challenge inputs, the consistency is trivially satisfied.

Challenge phase: Upon receiving challenge input $\mathbf{x}^*$, Ch computes the real DPRF evaluation on $\mathbf{x}^*$ as $f_{\{\mathbf{k}_{j,r}\}_{P_j \in \mathcal{P}'}}^{\text{dist}}(\mathbf{x}^*)$ for all $r \in [2, \text{rep}]$ and checks if all of them are equal. If they are equal, this value is referred

to as v (used in answering partial evaluation query on $\mathbf{x}^*$ in case of maximum corruption), and returned to Ad. Note that Ch's answer to Ad is necessarily the same as in earlier hybrids.

Lemma 5. *Ad can distinguish Hybrid_{3,1} from Hybrid₂ with advantage $\epsilon_{2,3,1}$ for some $\epsilon_{2,3,1} = \text{negl}(\lambda)$.*

Proof. We prove the phase-wise indistinguishability between Hybrid_{3,1} and Hybrid₂ in the view of Ad.

- *Partial evaluation query.* For $r \in [2, \text{rep}]$, the responses are identical to Hybrid₂. So, the focus is on indistinguishability of hybrids in case of $r = 1$. We discuss three different scenarios in the following:
 - Non-challenge input: Since the responses are of the form $[\langle \mathbf{A} \cdot \mathbf{s}_i, \mathbf{k}'_{j,1} \rangle]_{q_1}$, where $\mathbf{A} \cdot \mathbf{s}_i$ is statistically indistinguishable from uniform random over $\mathbb{Z}_q^n$ due to leftover hash lemma and $\mathbf{k}'_{j,1}$ is sampled uniformly at random from $\mathbb{Z}_q^n$, they belong to the same distribution as of real partial evaluations which are of the form $[\langle \mathcal{H}(\mathbf{x}_i), \mathbf{k}_{j,1} \rangle]_{q_1} = [\langle \mathbf{R} \cdot \mathbf{A} \cdot \mathbf{s}_i, \mathbf{k}_{j,1} \rangle]_{q_1}$ in Hybrid₂.
 - Challenge input (when corruption threshold is not reached): Here the responses are of the form $[\langle \mathbf{A} \cdot \mathbf{s}^*, \mathbf{k}''_{j,1} \rangle]_{q_1}$. Since $\mathbf{A} \cdot \mathbf{s}^*$ is statistically indistinguishable from uniform random over $\mathbb{Z}_q^n$ due to leftover hash lemma and $\mathbf{k}''_{j,1} = \mathbf{R}^{-1} \cdot \mathbf{k}'_{j,1}$ is uniform random over $\mathbb{Z}_q^n$ due to uniform randomness of both $\mathbf{R}$ and $\mathbf{k}'_{j,1}$, the simulated partial evaluation essentially comes from the same distribution as of real partial distribution which is of the form $[\langle \mathcal{H}(\mathbf{x}^*), \mathbf{k}_{j,1} \rangle]_{q_1} = [\langle \mathbf{A} \cdot \mathbf{s}^*, \mathbf{k}_{j,1} \rangle]_{q_1}$ for uniform random $\mathbf{k}_{j,1}$.
 - Challenge input (when corruption threshold is reached): The simulated partial evaluation of the form $[v \cdot \frac{q}{q_1} - \sum_{(\mathcal{P}', P_j) \in \mathcal{L}} \langle \mathbf{A} \cdot \mathbf{s}^*, \mathbf{k}''_{j,1} \rangle]_{q_1}$ is indistinguishable from real partial evaluations $[\langle \mathcal{H}(\mathbf{x}^*), \mathbf{k}_{j,1} \rangle]_{q_1}$ by Lemma 1 of [SPM24].
- *Corruption query:* For $r \in [2, \text{rep}]$, the responses to corruption queries are identical to that in Hybrid₂. For $r = 1$, the responses are simulated without using real shares as $\mathbf{k}''_{j,1} = \mathbf{R}^{-1} \cdot \mathbf{k}'_{j,1}$, where both $\mathbf{R}$ and $\mathbf{k}'_{j,1}$ are uniform random over $\mathbb{Z}_q^n$. Thus, they are uniform random over $\mathbb{Z}_q^n$, just similar to that of the real shares $\mathbf{k}_{j,1}$ in Hybrid₂.
- *Challenge phase:* In Hybrid₂, Ch computes the DPRF evaluation using real shares of some t -sized subset $\mathcal{P}'$ for each of rep different sets of shares, and responds with the result only if it is same in all rep cases. In Hybrid_{3,1}, Ch computes the DPRF evaluation using real shares of some t -sized subset $\mathcal{P}'$ for $(\text{rep} - 1)$ different sets of shares with $r \neq 1$, and then behaves identically. It is implied that for $r = 1$, we assume the DPRF evaluation to be same with the real DPRF evaluations, since we simulate the t^{th} partial evaluation on $\mathbf{x}^*$ using this value (v), in case of maximal corruption. Thus, Ch's response to challenge query is identical in both the hybrids.

Together, we conclude that Hybrid_{3,1} is statistically indistinguishable from Hybrid₂, and hence Ad has negligible distinguishing advantage, denoted by $\epsilon_{2,3,1}$, while transitioning from Hybrid₂ to Hybrid_{3,1}. $\square$

Next, we have a chain of hybrids represented as Hybrid_{3,r} for $r \in [2, \text{rep}]$. Broadly, **Hybrid_{3,r+1}** differs from **Hybrid_{3,r}** for all $r \in [1, \text{rep} - 1]$ in the following sense:

- *Partial evaluation query.* For each query of the form $(\mathbf{x}_i, P_j, \mathcal{P}')$, Ch provides Ad with rep partial evaluations in both the hybrids, each corresponding to one of rep different sets of shares. In both the hybrids, the first r evaluations are simulated without using real shares, and the last $(\text{rep} - r - 1)$ evaluations are computed using real shares. Thus, the two hybrids differ only in how Ch responds for $(r + 1)^{\text{th}}$ evaluation. In Hybrid_{3,r} it is real, whereas in Hybrid_{3,r+1} it is simulated using the same method as described in Hybrid_{3,1}.
- *Corruption query.* Analogously, in Hybrid_{3,r}, Ch sends real share $\mathbf{k}_{j,r+1}$ corresponding to $(r + 1)^{\text{th}}$ set of shares whereas in Hybrid_{3,r+1}, Ch responds with simulated share $\mathbf{k}''_{j,r+1}$.
- *Challenge phase.* The DPRF evaluation on challenge input $\mathbf{x}^*$ is decided based on $(\text{rep} - r)$ real evaluations in Hybrid_{3,r}, whereas in Hybrid_{3,r+1} it is based on $(\text{rep} - r - 1)$ real evaluations.

Now, we describe $\text{Hybrid}_{3,\text{rep}}$ in the following, where Ch does not make use of any of the real shares while responding to Ad's queries in all the phases, including the challenge phase. This hybrid can be thought of lying completely in the simulated world.

Hybrid_{3,rep}. Here, the random oracle is programmed in exactly the same manner as in Hybrid_2 (and remain unchanged through $\text{Hybrid}_{3,r}$ for all $r \in [\text{rep}]$). We describe all other interactions in the following.

- Partial evaluation query. Ch responds to a partial evaluation query of the form $(\mathbf{x}_i, P_j, \mathcal{P}')$ as,

$$[\lfloor \langle \mathbf{A} \cdot \mathbf{s}_i, \mathbf{k}'_{j,1} \rangle \rfloor_{q_1}, \dots, \lfloor \langle \mathbf{A} \cdot \mathbf{s}_i, \mathbf{k}'_{j,r} \rangle \rfloor_{q_1}, \dots, \lfloor \langle \mathbf{A} \cdot \mathbf{s}_i, \mathbf{k}'_{j,\text{rep}} \rangle \rfloor_{q_1}] \in \mathbb{Z}_{q_1}^{\text{rep}},$$

such that each $\mathbf{k}'_{j,r} \xleftarrow{\$} \mathbb{Z}_q^n$. In case of challenge input $\mathbf{x}^*$, if the corruption threshold is not reached by Ad, the response is,

$$[\lfloor \langle \mathbf{A} \cdot \mathbf{s}^*, \mathbf{k}''_{j,1} \rangle \rfloor_{q_1}, \dots, \lfloor \langle \mathbf{A} \cdot \mathbf{s}^*, \mathbf{k}''_{j,r} \rangle \rfloor_{q_1}, \dots, \lfloor \langle \mathbf{A} \cdot \mathbf{s}^*, \mathbf{k}''_{j,\text{rep}} \rangle \rfloor_{q_1}] \in \mathbb{Z}_{q_1}^{\text{rep}},$$

such that $\mathbf{k}''_{j,r} = \mathbf{R}^{-1} \cdot \mathbf{k}'_{j,r}$ holds for already sampled $\mathbf{k}'_{j,r}$. However, when the maximal corruption threshold is reached for some subset $\mathcal{P}'$, the t^{th} honest partial evaluation on $\mathbf{x}^*$ is simulated as $\mu_{\text{sim},r} = \lfloor v_{\text{sim}} \cdot \frac{q}{q_1} - \sum_{(\mathcal{P}', P_j) \in \mathcal{L}} \langle \mathbf{A} \cdot \mathbf{s}^*, \mathbf{k}''_{j,r} \rangle \rfloor_{q_1}$, where v_{sim} is the response of Ch in the challenge phase. Thus, the response becomes,

$$[\mu_{\text{sim},1}, \dots, \mu_{\text{sim},r}, \dots, \mu_{\text{sim},\text{rep}}] \in \mathbb{Z}_{q_1}^{\text{rep}}.$$

- Corruption query. In response to any corruption query of the form $(\mathcal{P}', P_j)$, Ch returns all rep simulated shares, i.e., $[\mathbf{k}''_{j,1}, \dots, \mathbf{k}''_{j,r}, \dots, \mathbf{k}''_{j,\text{rep}}] \in \mathbb{Z}_q^{n \times \text{rep}}$.
- Challenge phase. Ch samples $v_{\text{sim}} \xleftarrow{\$} \mathbb{Z}_p$ and sends it as response to challenge query $\mathbf{x}^*$.

Lemma 6. Ad distinguishes $\text{Hybrid}_{3,\text{rep}}$ from Hybrid_2 with advantage $(\text{rep} - 1) \cdot \epsilon_{2,3.1} + \epsilon'$, where $\epsilon' = \text{negl}(\lambda)$.

Proof. Since the transition from $\text{Hybrid}_{3,r}$ to $\text{Hybrid}_{3,r+1}$ for all $r \in [\text{rep} - 1]$ is exactly similar to the transition from Hybrid_2 to $\text{Hybrid}_{3,1}$, and each transition introduces a distinguishing advantage of $\epsilon_{2,3.1}$ for Ad, it gives a total distinguishing advantage of $(\text{rep} - 1) \cdot \epsilon_{2,3.1}$ between Hybrid_2 and $\text{Hybrid}_{3,\text{rep}-1}$ for Ad.

The transition from $\text{Hybrid}_{3,\text{rep}-1}$ to $\text{Hybrid}_{3,\text{rep}}$ is similar to that of the transition between any $\text{Hybrid}_{3,r}$ and $\text{Hybrid}_{3,r+1}$, for $r \in [\text{rep} - 1]$ with respect to the partial evaluation query phase and the corruption query phase. However, in challenge phase the transition is different. Ch replaces the real DPRF evaluation with a uniform random value in $\mathbb{Z}_p$. Note that by the correctness of DPRF, the real DVRF evaluation in $\text{Hybrid}_{3,\text{rep}-1}$ is equal to that of base PRF evaluation which is $f_{\mathbf{k}}^{\text{base}}(\mathcal{H}(\mathbf{x}^*)) = \lfloor \langle \mathcal{H}(\mathbf{x}^*), \mathbf{k} \rangle \rfloor_p$, with $\mathbf{k}$ being the master secret key. And by the LWR assumption, this value is computationally indistinguishable from a uniform random value v_{sim} in $\text{Hybrid}_{3,\text{rep}}$. We assume Ad's distinguishing advantage to be ϵ' for some $\epsilon' = \text{negl}(\lambda)$ while transitioning from $\text{Hybrid}_{3,\text{rep}-1}$ to $\text{Hybrid}_{3,\text{rep}}$. $\square$

Together, we conclude that Ad can distinguish between Hybrid_2 and $\text{Hybrid}_{3,\text{rep}}$ with advantage $(\text{rep} - 1) \cdot \epsilon_{2,3.1} + \epsilon'$.

Next, we have $(\text{rep} - 1)$ hybrids represented as $\text{Hybrid}_{4,r}$ in decreasing order of r for $r \in [\text{rep} - 1]$. Each $\text{Hybrid}_{4,r}$ is identical to $\text{Hybrid}_{3,r}$ for all $r \in [\text{rep} - 1]$, except in challenge phase. In $\text{Hybrid}_{3,r}$, Ch responds with real DPRF evaluation whereas in $\text{Hybrid}_{4,r}$, Ch responds with a uniform random value in $\mathbb{Z}_p$. The indistinguishability while transitioning from $\text{Hybrid}_{4,r+1}$ to $\text{Hybrid}_{4,r}$ can be argued in the similar manner as we do in case of transitioning from $\text{Hybrid}_{3,r}$ to $\text{Hybrid}_{3,r+1}$. The responses to partial evaluation queries and corruption queries are transitioned from simulated to real for one of the rep sets of shares, one at a time for each hybrid transition.

To summarize, Ad can distinguish $\text{Hybrid}_{4,1}$ from $\text{Hybrid}_{3,\text{rep}}$ with advantage $(\text{rep} - 1) \cdot \epsilon_{2,3.1}$.

Next, for the sake of completeness, we have three more hybrids **Hybrid₅**, **Hybrid₆**, **Hybrid₇**, which are identical to Hybrid₂, Hybrid₁ and Hybrid₀, respectively, except the fact that in the challenge phase, Ch responds with uniform random values in $\mathbb{Z}_p$ in the former cases and with real DPRF evaluations in the later.

Note that the interactions between Ch and Ad in the final hybrid Hybrid₇ lie in the real world, but the response in the challenge phase is a uniform random value. Ad’s views in the first and last hybrid (i.e., Hybrid₀ and Hybrid₇) emulate the notion of pseudorandomness of a DPRF under adaptive corruption, as given in Section 2.2. Since for a specific choice of parameters, $\text{rep} = O(1)$ is a constant, Ad can distinguish Hybrid₇ from Hybrid₀ with advantage,

$$\frac{1}{Q+1} \left(2\epsilon_{0,1} + 2\epsilon_{1,2} + (2 \cdot \text{rep} - 1) \cdot \epsilon_{2,3,1} + \epsilon' \right) = \text{negl}(\lambda).$$

The multiplicative factor of $\frac{1}{Q+1}$ arises as Ch’s guess of the challenge input $\mathbf{x}^*$ is correct with probability $\frac{1}{Q+1}$. This concludes the proof of Theorem 3, thereby, establishing the adaptive security of Construction 1 in the ROM. □

3.3.2 Adaptive Security of LWR-based DPRF [SPM24] in the QROM

We now argue that Construction 1 remains adaptively secure even when the hash function is modeled as a quantum-accessible ROM (QROM). Concretely, we state and prove the following theorem.

Theorem 4 (Adaptive Security of LWR-based DPRF in the QROM). *Assuming the hardness of LWR, Construction 1 is adaptively secure in the quantum random oracle model (QROM).*

Proof. Though the adaptive proof of security of LWR-based DPRF in [SPM24] in ROM can not immediately be imported to QROM settings, where Ad can query the random oracle (RO) in superposition, we prove that with slight modification in the proof it is possible to argue adaptive security of the DPRF even in QROM.

Before proceeding further, we emphasize that the concept of lazy sampling in ROM does not work in QROM. This is because Ad can query RO in superposition of exponential number of inputs with varying amplitudes. So, the RO (i.e., the function $\mathcal{H}(\cdot)$) must be (conceptually) initialized with the complete input-output behavior in the beginning of the game. Note that only this conceptual shift suffices to prove the *static* security of LWR-based DPRF in QROM in [SPM24]¹.

However that is not the case for the proof of adaptive security, because the proof heavily relies on the programmability of the RO. Again, unlike classical ROM, programming the RO output at the unqueried inputs is not applicable in QROM, because there is no concept of “unqueried input” in QROM due to superpositioned queries, and hence this manipulation can be detected by Ad. Note that though there are a total of $(2 \cdot \text{rep} + 5)$ hybrids in the proof of adaptive security in ROM, the RO description is modified only during transition from Hybrid₀ to Hybrid₁, Hybrid₁ to Hybrid₂ (and analogously during transition from Hybrid₅ to Hybrid₆, Hybrid₆ to Hybrid₇). Since RO’s response remains identical throughout all the other hybrids, Ad does not have any extra advantage to distinguish between them by exploiting its quantum access to RO. So, we focus on two following transitions:

- Hybrid₀ $\rightarrow$ Hybrid₁: Since, $(\mathbf{R} \cdot \mathbf{A}) \cdot \mathbf{s}_i$ is statistically close to $\mathcal{H}(\mathbf{x}_i)$, these two hybrids are indistinguishable in the view of a quantum polynomial time (QPT) adversary.
- Hybrid₁ $\rightarrow$ Hybrid₂: Considering that the RO is initialized in the beginning of the game in both hybrids, note that the RO in these two hybrids differ only at a single input, namely $\mathbf{x}^*$. Distinguishing between these two instantiations of RO is equivalent to the problem of searching the differing input position (which is $\mathbf{x}^*$) using polynomial number of superpositioned RO queries. Ad does not know $\mathbf{x}^*$ at the

¹In particular, in the proof of static security of [SPM24] no RO programming is involved and hence the RO’s response to Ad’s queries remain necessarily same throughout all the four hybrids, thus providing no extra advantage to Ad in distinguishing between those hybrids, even with quantum access to the RO.

time of RO initialization; because Ch guesses the challenge input beforehand and aborts the game later if the guess is wrong. Then by [BBBV97], a QPT adversary has a distinguishing advantage of $O(\frac{Q'^2}{q^n})$, where $Q' = \text{poly}(\lambda)$ is the total number of RO queries.

This concludes the proof of Theorem 4, thereby, establishing the adaptive security of Construction 1 in the QROM. $\square$

3.3.3 Adaptive Security of MLWR-based DPRF in the QROM

Finally, we state and prove the main security theorem, which captures the adaptive security of our proposed MLWR-based DPRF in the QROM.

Theorem 5 (Adaptive Security of MLWR-based DPRF in the QROM). *Assuming the hardness of MLWR, Construction 2 is adaptively secure in the QROM.*

Proof. The proof approach closely follows the adaptive security proof, previously outlined for the LWR-based DPRF, except with a few differences.

An alternative of leftover hash lemma in module settings: We use the leftover hash lemma [Reg09] as an important tool to simulate the RO in $\text{Hybrid}_{\text{sim}}$. Analogously, to program the random oracle in module setting, we need a mechanism to output a uniform random element of $\mathcal{R}_{N,q}^d$. We resort to the following regularity lemma from [LPR13].

Lemma 7 (Restated from Lemma 2). *Let $\mathbf{A} = [\mathbf{I}_d | \tilde{\mathbf{A}}] \leftarrow \mathcal{R}_{N,q}^{d \times \ell}$, such that $\tilde{\mathbf{A}} \xleftarrow{\$} \mathcal{R}_{N,q}^{d \times (\ell-d)}$ is uniformly random and $\mathbf{I}_d$ is the identity matrix. Then with probability $1 - 2^{-\Omega(N)}$ over the choice of $\tilde{\mathbf{A}}$, the distribution of $\mathbf{A} \cdot \mathbf{s} \in \mathcal{R}_{N,q}^d$, where each element of $\mathbf{s} \in \mathcal{R}_{N,q}^\ell$ is sampled from a discrete Gaussian distribution of radius $r > 2N \cdot q^{\frac{d}{\ell} + \frac{2}{N\epsilon}}$ over $\mathcal{R}$, is within statistical distance $2^{-\Omega(N)}$ of the uniform distribution over $\mathcal{R}_{N,q}^d$.*

The lemma implies that, $\mathbf{A} \cdot \mathbf{s} \in \mathcal{R}_{N,q}^d$ can be used to program the output of random oracle if $\mathbf{A}$ and $\mathbf{s}$ come from the above-mentioned distributions. Also, we replace the matrix $\mathbf{R}$ of LWR setting with a scalar $r \xleftarrow{\$} \mathcal{R}_{N,q}$ in MLWR setting such that r is invertible modulo $(x^N + 1)$. Note that with $\mathbf{A} = [\mathbf{I}_d | \tilde{\mathbf{A}}] \in \mathcal{R}_{N,q}^{d \times \ell}$, $r \cdot \mathbf{A}$ performs scalar multiplication on each element of $\mathbf{A}$ with r , and the output is $[\mathbf{I}'_d | r \cdot \tilde{\mathbf{A}}]$. Here, $\mathbf{I}'_d$ is a diagonal matrix with each diagonal element initialized to r . From [LPR13], we conclude that $r \cdot \mathbf{A}$ preserves the matrix format that is eligible for regularity lemma to be applied, i.e., $(r \cdot \mathbf{A}) \cdot \mathbf{s}$ is uniform random over $\mathcal{R}_{N,q}^d$ for suitable distribution of $\mathbf{s}$. The rest of the proof essentially proceeds in a similar manner.

Concluding the proof. Replacing the leftover hash lemma with the regularity lemma from [LPR13] and a few obvious changes in dimensions and parameters translate the proof of adaptive security of LWR-based DPRF into a proof for Construction 2. $\square$

3.4 Choice of Parameters for (M)LWR-based Adaptive DPRF

In this section, we summarize the concrete choice of parameters for both the LWR and MLWR-based DPRFs such that adaptive security holds. Notably, in the partial evaluation phase of these adaptive DPRFs, the rounding step transitions values from modulus q to modulus q_1 ; hence, the error here is of the order of $\frac{q}{q_1}$. In other words, the modulus-to-noise ratio is of the order $\frac{q}{e} = \frac{q}{q_1} = q_1$. The solution to average-case LWR problem with a smaller modulus-to-noise ratio implies a solution to worst-case lattice problem with a smaller approximation factor. Hence, the smaller we keep the modulus-to-noise ratio, the harder it is to solve the underlying worst-case lattice problem. While choosing concrete parameters for practical instantiations, our objective is to ensure that the modulus-to-noise ratio $\frac{q}{e}$ remains sufficiently small to mitigate any known attack strategies. We further elaborate on the concrete choice of parameters for both the instantiations of adaptive DPRFs in the following.

Parameters for LWR-based Adaptive DPRF. The following parameter set for the LWR-based DPRF, as suggested by [SPM24], ensures 128-bit security in static corruption scenarios: $n = 512$, $q = 2^{32}$, $q_1 = 2^{28}$, $p = 2^{10}$. By proof of Theorem 3, the exact same parameter set should also provide adaptive security. However, in this work, we use a different parameter set, namely, $n = 1024$, $q = 2^{64}$, $q_1 = 2^{38}$, $p = 2^{10}$, while providing experimental results for adaptively secure LWR-based DPRF. Intuitively, the modulus q , the error proportional to $\frac{q}{q_1}$, and the dimension n together correspond to the security of DPRF whereas the ratio $\frac{q_1}{p}$ corresponds to the correctness of DPRF. Our concrete choice of parameters provides at least 128 bits of security considering the attacks listed in *lattice estimator*¹ [APS15].

Parameters for MLWR-based Adaptive DPRF. We use the following parameter set for MLWR-based DPRF: $N = 256$, $d = 4$, $q = 2^{64}$, $q_1 = 2^{38}$, $p = 2^{10}$ to provide at least 128 bits of security. The reasons behind choosing this parameter set for our construction are the following: (i) there are currently no known attack on MLWE with these parameters, which are actually comparable to those used in Kyber [BDK⁺18] (ii) there are no meaningful attack on MLWR that are better than those on MLWE (in the same way as there are no meaningful attack on LWR that are better than LWE attacks), and (iii) there are no meaningful attack on RLWE or MLWE that are better than LWE attacks [ACC⁺21].

4 EQuADiSE

In this section, we present EQuADiSE (Efficient Quantum-safe Adaptive DiSE), which builds upon Construction 2 in Section 3.2. EQuADiSE is the first practically efficient, adaptively secure, and plausibly post-quantum secure construction of DiSE [AMMR18] that incurs linear (in the number of parties) encryption/decryption overheads.

4.1 Construction Overview

In this subsection, we present an overview of our construction of EQuADiSE, which follows prior approaches for constructing DiSE from DPRF [AMMR18, Muk20, SPM24]. For the sake of completeness, we first present some background material on DiSE.

Informally speaking, DiSE enables the distributed computation of both encryption and decryption in a symmetric encryption scheme, i.e., encryption and decryption are performed by multiple servers in a distributed manner. More concretely, in (t, T) -DiSE for $t < T$, whenever a plaintext (or ciphertext) needs to be encrypted (or decrypted), a service request is initiated, and any t out of T encryption (or decryption) servers are contacted to process the request. Each selected server performs a partial computation using its individual key share and contributes to the formation of a message-specific symmetric key in a distributed fashion. This symmetric key is then used as the encryption (or decryption) key for the actual plaintext (or ciphertext).

Template for DiSE from DPRF. We recall below the template for realizing DiSE from any DPRF. This template was first introduced in [AMMR18], and has subsequently been adopted in [Muk20, SPM24] (which we summarize in Table 2). We follow the same approach to realize EQuADiSE based on our MLWR-based adaptively secure DPRF from Section 3.2.

Let $\mathcal{P} = \{P_j : j \in [T]\}$ be the set of encryption/decryption servers. Given the following cryptographic primitives as its building blocks:

- A DPRF $\text{DPRF} = (\text{DPRF.Setup}, \text{DPRF.PartialEval}, \text{DPRF.FinalEval})$,
- A PRG with polynomial stretch, and
- A commitment scheme $\text{C} = (\text{C.Setup}, \text{C.Com})$.

¹<https://github.com/malb/lattice-estimator/commit/a51a410192e39d7df133c775bbe058ecacb5577d>

DiSE variants	Underlying DPRF	Key Share Size (in bits)	Distributed Encryption Overheads	Communication Overheads	Post-quantum?	Adaptively secure?
Original DiSE [AMMR18]	AES(128)-based	$128 \times \binom{T-1}{t-1}$	$O(\binom{T-1}{t-1})$	$O(\binom{T}{t})$	No	Yes
Original DiSE [AMMR18]	DDH-based	256×1	$O(t)$	$O(t)$	No	No
Adaptive DiSE [Muk20]	Adaptive DDH-based	512×1	$O(t)$	$O(t)$	No	Yes
PQDiSE [SPM24]	LWR-based	$512 \times 32 \times \binom{T-1}{t-1}$	$O(t)$	$O(t)$	Yes	No
EQuADiSE	MLWR-based	$(256 \times 4) \times 64 \times \binom{T-1}{t-1}$	$O(t)$	$O(t)$	Yes	Yes

Table 2: Comparison of DiSE variants implying EQuADiSE to be only efficient DiSE that is both post-quantum and adaptively secure

one can realize a DiSE construction as described in Construction 3,

$$\text{DiSE} = (\text{DiSE.Setup}, \text{DiSE.DistEncrypt}, \text{DiSE.DistDecrypt}).$$

Construction 3 (DiSE from DPRF, adapted from [AMMR18]). *The construction is as follows:*

- **DiSE.Setup:** $\text{DPRF.Setup}(1^\lambda, t, T)$ provides DPRF key shares k_j to $P_j \forall j \in [T]$ and $\text{C.Setup}(1^\lambda)$ outputs public parameters for the commitment scheme.
- **DiSE.DistEncrypt:** An encryptor E encrypts a plaintext m as follows. It first computes a commitment α on the message m using randomness ρ , and then sends it to t encryption servers. Each server performs partial evaluation on α using its secret share k_j and computes μ_j . On receiving all t μ_j 's, E combines them to generate symmetric key $\tilde{k}$, which is used to encrypt m . The ciphertext has the form $c = (\alpha, e)$, where $e = \text{PRG}(\tilde{k}) \oplus (m || \rho)$.
- **DiSE.DistDecrypt:** A decryptor D , when provided with a ciphertext $c = (\alpha, e)$, contacts a set of t decryption servers and provides them with α . Each server computes partial evaluation μ_j on α , and sends it to D . D combines them all to generate the symmetric key $\tilde{k}$. Now $(m || \rho)$ is retrieved from e , by XORing it with $\text{PRG}(\tilde{k})$. Plaintext m is obtained by discarding the randomness ρ .

By plugging our MLWR-based adaptively secure DPRF from Section 3.2 into the above template, we obtain EQuADiSE. In Table 2, we compare EQuADiSE with other known instantiations of the above template, summarized below:

- Two instances of DiSE were proposed in [AMMR18] based on: (i) the DDH-based static DPRF from [NPR99] (this is not adaptively secure), and (ii) the AES-128 based (combinatorially constructed) DPRF from [NPR99] (this is only adaptively secure for (t, T) such that $\binom{T}{t}$ is polynomial).
- An adaptive instance of DiSE was proposed in [Muk20] based on an adaptively secure extension of the DDH-based DPRF from [NPR99] (this is quantum-unsafe).
- PQDiSE [SPM24] is based on a static LWR-based DPRF construction (this does not offer adaptive security based on the analysis in [SPM24]).

We restrict the comparison in Table 2 to precisely those works that actually propose concrete instantiations of DiSE based on DPRFs. While it is possible to instantiate the above template based on other known constructions of DPRFs (summarized in Appendix A), we defer the comparison to the experimental evaluation in Section 5.

5 Experimental Evaluation

In this section, we experimentally evaluate and compare the performance of our MLWR-based DPRF in terms of evaluation time, relative to several existing DPRFs. We also experimentally evaluate and compare the performance of EQuADiSE with several known instances of DiSE that use DPRFs as building blocks. Our implementation is available here ¹. Throughout the section, the x -axis of all graphs presents the total number of parties T in $(\frac{T}{2}, T)$ -DPRFs. The y -axis of all graphs is presented on a logarithmic scale.

¹<https://github.com/SayaniSinha97/EQuADiSE>

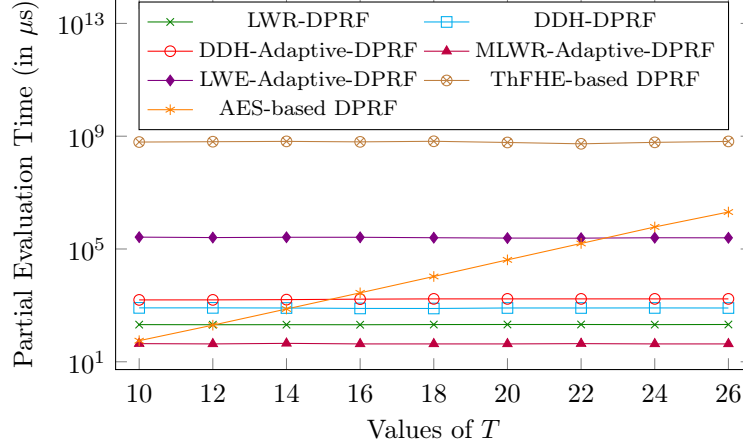


Figure 1: Comparison of the partial evaluation latency across various $(\frac{T}{2}, T)$ -DPRFs

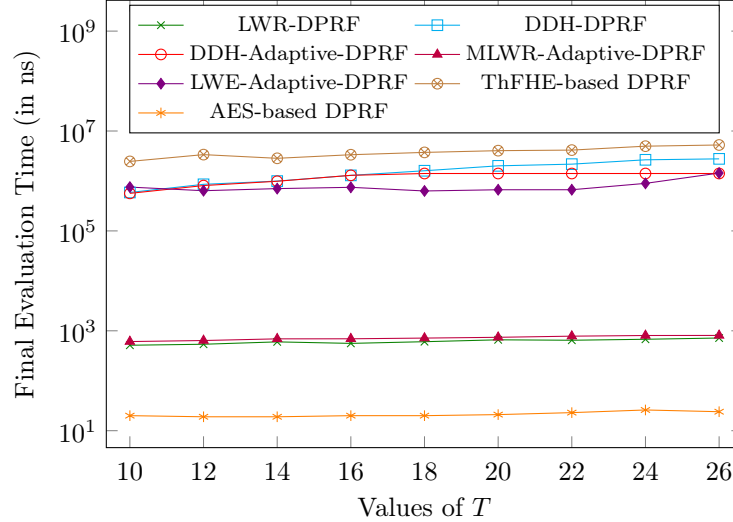


Figure 2: Comparison of the final evaluation latency (equivalently, the time to combine the partial evaluations) across various $(\frac{T}{2}, T)$ -DPRFs

Experimental Platform. All experiments were executed on a high-end server with an Intel(R) Xeon(R) Silver 4214R CPU (2.40GHz clock frequency), 48 cores, 128GB RAM. We use NTL library¹ to support arithmetic over $\mathbb{Z}_q^n$ and $\mathcal{R}_{N,q}$. We use Blake2² to instantiate the hash function.

Comparison across DRPFs. We present a comprehensive comparison of the performance of our MLWR-based DPRF with: (i) the AES-based combinatorial DPRF from [AMMR18], (ii) the DDH-based DPRF from [AMMR18], (iii) the adaptive DDH-based DPRF from [Muk20], (iv) the adaptive post-quantum LWE-based DPRF from [LST21], (v) the static LWR-based DPRF from [SPM24], and (vi) the static ThFHE-based DPRF utilizing the *Universal Thresholdizer* (UT) tool [BGG⁺18], all within semi-honest adversarial settings. We implement the ThFHE-based DPRF using the TFHE library³ [CGGI20], with AES as the underlying PRF. For the ThFHE implementation, we adopt the parameter choices recommended by [CSS⁺22].

Comparison of Partial Evaluation Latency [Figure 1]. Since each party would typically compute its own

¹<https://libnt1.org/>

²<https://www.blake2.net/>

³<https://tfhe.github.io/tfhe/>

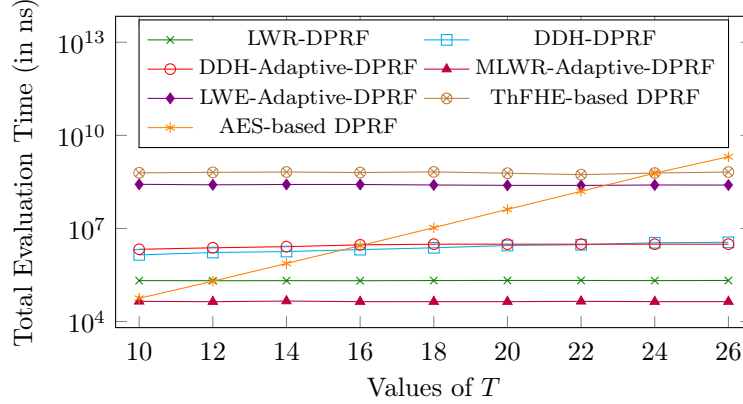


Figure 3: Comparison of the total evaluation latency across various $(\frac{T}{2}, T)$ -DPRFs

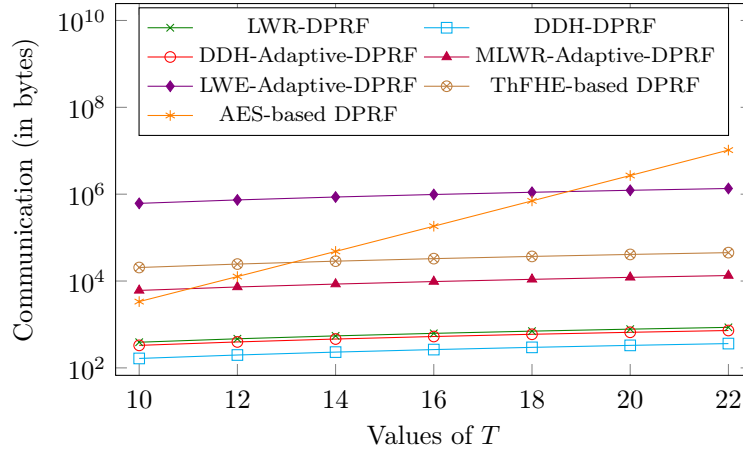


Figure 4: Comparison of the total communication overheads across various $(\frac{T}{2}, T)$ -DPRFs

partial evaluation in parallel, we plot the maximum partial evaluation latency in Figure 1. The notably poor performance of ThFHE-based DPRF [BGG⁺18] results from the very large computational overhead of homomorphically evaluating the AES circuit. On the other hand, for the AES-based combinatorial DPRF [NPR99], the partial evaluation latency increases exponentially (appearing linear in the log-scale plot) with larger T , as the maximum partial evaluation latency is proportional to $\binom{T-1}{t-1}$. This differs from other DPRFs, where the computational load of partial evaluation is evenly distributed across the t participants, with individual partial evaluation latencies being independent of (t, T) .

The adaptive DDH-based DPRF from [Muk20] incurs higher partial evaluation latency than its non-adaptive counterpart from [NPR99]. This is because the adaptively secure version requires two modular exponentiations within a DDH-hard group, rather than one in the non-adaptively secure version. Finally, partial evaluation of the LWE-based DPRF from [LST21] involves multiple matrix multiplications, which requires a logarithmic depth circuit, and thus leads to a significantly high partial evaluation latency.

Overall, our experiments that the proposed MLWR-based adaptive DPRF exhibits the lowest partial evaluation latency across almost all values of T . In particular, it is faster than the LWR-based static DPRF from [SPM24] on average. This is because each evaluation of the MLWR-based DPRF generates a total of (256×10) pseudorandom bits (since $N = 256$, $p = 2^{10}$). In contrast, a single LWR-based DPRF evaluation in [SPM24] generates only 10 bits, which has to be sequentially amplified subsequently.

Comparison of Final Evaluation Latency [Figure 2]. Figure 2 plots the time required to finally combine the various partial evaluations in each DPRF. This is naturally the smallest for the AES-based combinatorial

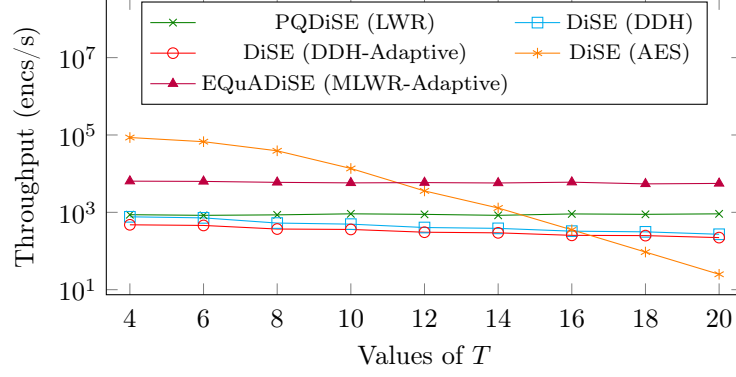


Figure 5: Comparison of online throughput (encryptions per second) across various $(\frac{T}{2}, T)$ -DiSE instances

DPRF from [NPR99], since it simply involves bit-wise XORing the partial evaluations.

The plot lines for the LWR-based static DPRF from [SPM24] and our adaptive MLWR-based DPRF overlap closely. This is because the final evaluation consists of similar operations (addition over a ring followed by rounding off) in both cases. Both of these DPRFs incur smaller final evaluation latencies than the DDH-based [NPR99, Muk20], the LWE-based [LST21] and the ThFHE-based [BGG⁺18] DPRFs. In particular, the final evaluation procedure for the LWE-based DPRF from [LST21] consists of $O(n \log q)$ rounding-off operations followed by a deterministic randomness extractor function. This causes it to incur significantly higher final evaluation latency than both the LWR-based DPRF from [SPM24] and our MLWR-based DPRF.

Comparison of Total Evaluation Latency [Figure 3]. Finally, Figure 3 plots the total (partial+final) evaluation time (in nanoseconds) across all of the DPRFs. Note that the latencies reported in Figure 1 and Figure 2 are represented microseconds and nanoseconds, respectively (depicted on log scale), while the total evaluation latencies reported in Figure 3 are again represented in nanoseconds (again on log scale). Evidently, our MLWR-based DPRF exhibits the lowest total evaluation latency. This is along the lines of Figure 1, since the contribution from the partial evaluation latency dominates the total evaluation latency across all DPRFs.

Comparison of Total Communication Overhead [Figure 4]. Figure 4 compares the concrete communication overheads of the considered DPRF instantiations. Let $P_1, \dots, P_t$ denote the t collaborating parties participating in a (t, T) -DPRF evaluation. The communication cost is dominated by the transmission of partial evaluations from the participating parties to a designated combiner during the partial evaluation phase. Specifically, the total communication overhead is given by $\text{cnt} \cdot |\mu|$, where $|\mu|$ denotes the size of a partial evaluation corresponding to a single secret share, and cnt denotes the total number of partial evaluations communicated.

The results show that our MLWR-based DPRF incurs communication overhead that scales linearly with the number of parties, in contrast to the exponential growth exhibited by the AES-based DPRF. In comparison with lattice-based adaptively secure DPRFs, the MLWR-based DPRF incurs higher communication overhead than the LWR-based DPRF, but lower communication overhead than the LWE-based DPRF.

Comparison across DiSE Instances. We finally compare EQuADiSE with instances of DiSE obtained by plugging the above DPRF constructions into the DPRF-based template from [AMMR18]. Since the LWE-based adaptive DPRF from [LST21] and ThFHE-based DPRF from [BGG⁺18] perform significantly worse than the other DPRFs (ref. Figure 3), we exclude them from this comparison.

Figure 5 compares the online throughput (number of encryptions per second) across the remaining DiSE instances obtained from the following DPRFs: (i) the non-adaptive DDH-based DPRF from [NPR99], (ii) the AES-based combinatorial DPRF from [NPR99], (iii) the DDH-based adaptive DPRF from [Muk20], (iv) the LWR-based DPRF from [SPM24], and (v) our MLWR-based adaptive DPRF. Here, (i) and (ii) yield instances of DiSE reported in [AMMR18], (iii) yields an instance of DiSE reported in [Muk20], (iv) yields PQ-DiSE [SPM24], while (v) yields our EQuADiSE construction.

Figure 5 illustrates that EQuADiSE achieves the highest online throughput among all of the above schemes. This follows naturally from the fact that our MLWR-based DPRF achieves the lowest total evaluation latency (as per Figure 3).

In particular, we highlight that EQuADiSE achieves higher online throughput than PQ-DiSE [SPM24], even though PQ-DiSE only achieves static security. This is because a single evaluation of our MLWR-based DPRF generates 256×10 pseudorandom bits, which enables the encryption of twenty 128-bit plaintexts in a single batch. In contrast, the LWR-based DPRF from [SPM24] requires as many as 13 evaluations to encrypt a single 128-bit plaintext, as each evaluation only produces 10 pseudorandom bits.

6 Discussion and Future Work

In this paper, we introduced a Module LWR-based distributed pseudorandom function (DPRF) that enjoys adaptive security in the QROM. Our construction achieves both online evaluation time and communication complexity linear in the number of participating parties. We implemented a prototype of the proposed DPRF and experimentally demonstrated that it achieves lower evaluation latency than existing DPRF constructions for threshold set sizes of practical interest.

Our proposed DPRF yields a realization of distributed symmetric-key encryption (DiSE) [AMMR18], which we call EQuADiSE. EQuADiSE is the first practically efficient, adaptively secure, and plausibly post-quantum instance of DiSE that incurs linear (in the number of parties) encryption/decryption overheads. We experimentally demonstrate that EQuADiSE achieves the highest online throughput among all known DiSE instantiations, particularly for larger number of parties.

A limitation of our Module LWR-based DPRF (and EQuADiSE) is that the key storage overhead per party is potentially exponential in the total number of parties. We leave it as an interesting open question to design a post-quantum, adaptively secure DPRF that preserves the efficient online throughput of our construction while requiring linear (or even sub-exponential) key storage overheads. Another promising direction is to augment the DPRF with *verifiable* partial evaluations, thus achieving malicious security. Such advances would not only strengthen the efficiency and security guarantees of the DPRF itself, but would also yield more practically efficient and robustly post-quantum secure instances of DiSE.

References

- [ACC⁺21] Martin Albrecht, Melissa Chase, Hao Chen, Jintai Ding, Shafi Goldwasser, Sergey Gorbunov, Shai Halevi, Jeffrey Hoffstein, Kim Laine, Kristin Lauter, et al. Homomorphic encryption standard. *Protecting privacy through homomorphic encryption*, pages 31–62, 2021.
- [ADL⁺22] Shashank Agrawal, Wei Dai, Atul Luykx, Pratyay Mukherjee, and Peter Rindal. Paradise: efficient threshold authenticated encryption in fully malicious model. In *International Conference on Cryptology in India*, pages 26–51. Springer, 2022.
- [AJLA⁺12] Gilad Asharov, Abhishek Jain, Adriana López-Alt, Eran Tromer, Vinod Vaikuntanathan, and Daniel Wichs. Multiparty computation with low communication, computation and interaction via threshold fhe. In *Advances in Cryptology—EUROCRYPT 2012: 31st Annual International Conference on the Theory and Applications of Cryptographic Techniques, Cambridge, UK, April 15–19, 2012. Proceedings 31*, pages 483–501. Springer, 2012.
- [AMMR18] Shashank Agrawal, Payman Mohassel, Pratyay Mukherjee, and Peter Rindal. Dise: distributed symmetric-key encryption. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, pages 1993–2010, 2018.
- [APS15] Martin R Albrecht, Rachel Player, and Sam Scott. On the concrete hardness of learning with errors. *Journal of Mathematical Cryptology*, 9(3):169–203, 2015.

- [BBBV97] Charles H Bennett, Ethan Bernstein, Gilles Brassard, and Umesh Vazirani. Strengths and weaknesses of quantum computing. *SIAM journal on Computing*, 26(5):1510–1523, 1997.
- [BDF⁺11] Dan Boneh, Özgür Dagdelen, Marc Fischlin, Anja Lehmann, Christian Schaffner, and Mark Zhandry. Random oracles in a quantum world. In *International conference on the theory and application of cryptography and information security*, pages 41–69. Springer, 2011.
- [BDK⁺18] Joppe Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-kyber: a cca-secure module-lattice-based kem. In *2018 IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 353–367. IEEE, 2018.
- [BGG⁺18] Dan Boneh, Rosario Gennaro, Steven Goldfeder, Aayush Jain, Sam Kim, Peter MR Rasmussen, and Amit Sahai. Threshold cryptosystems from threshold fully homomorphic encryption. In *Advances in Cryptology–CRYPTO 2018: 38th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19–23, 2018, Proceedings, Part I 38*, pages 565–596. Springer, 2018.
- [BGI15] Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing. In *Annual international conference on the theory and applications of cryptographic techniques*, pages 337–367. Springer, 2015.
- [BGM⁺16] Andrej Bogdanov, Siyao Guo, Daniel Masny, Silas Richelson, and Alon Rosen. On the hardness of learning with rounding over small modulus. In *Theory of Cryptography Conference*, pages 209–224. Springer, 2016.
- [BGV14] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. (leveled) fully homomorphic encryption without bootstrapping. *ACM Transactions on Computation Theory (TOCT)*, 6(3):1–36, 2014.
- [BPR12] Abhishek Banerjee, Chris Peikert, and Alon Rosen. Pseudorandom functions and lattices. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 719–737. Springer, 2012.
- [BS23] Katharina Boudgoust and Peter Scholl. Simple threshold (fully homomorphic) encryption from LWE with polynomial modulus. In Jian Guo and Ron Steinfeld, editors, *Advances in Cryptology - ASIACRYPT 2023 - 29th International Conference on the Theory and Application of Cryptology and Information Security, Guangzhou, China, December 4-8, 2023, Proceedings, Part I*, volume 14438 of *Lecture Notes in Computer Science*, pages 371–404. Springer, 2023.
- [CGGI20] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. Tfhe: fast fully homomorphic encryption over the torus. *Journal of Cryptology*, 33(1):34–91, 2020.
- [CGMS21] Mihai Christodorescu, Sivanarayana Gaddam, Pratyay Mukherjee, and Rohit Sinha. Amortized threshold symmetric-key encryption. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, pages 2758–2779, 2021.
- [CSS⁺22] Siddhartha Chowdhury, Sayani Sinha, Animesh Singh, Shubham Mishra, Chandan Chaudhary, Sikhar Patranabis, Pratyay Mukherjee, Ayantika Chatterjee, and Debdeep Mukhopadhyay. Efficient threshold fhe with application to real-time systems. *Cryptology ePrint Archive*, 2022.
- [DDEK⁺23] Morten Dahl, Daniel Demmler, Sarah El Kazdadi, Arthur Meyre, Jean-Baptiste Orfila, Dragos Rotaru, Nigel P Smart, Samuel Tap, and Michael Walter. Noah’s ark: Efficient threshold-fhe using noise flooding. In *Proceedings of the 11th Workshop on Encrypted Computing & Applied Homomorphic Cryptography*, pages 35–46, 2023.
- [Des94] Yvo G Desmedt. Threshold cryptography. *European Transactions on Telecommunications*, 5(4):449–458, 1994.

- [DKSRV18] Jan-Pieter D’Anvers, Angshuman Karmakar, Sujoy Sinha Roy, and Frederik Vercauteren. Saber: Module-lwr based key exchange, cpa-secure encryption and cca-secure kem. In *Progress in Cryptology–AFRICACRYPT 2018: 10th International Conference on Cryptology in Africa, Marrakesh, Morocco, May 7–9, 2018, Proceedings 10*, pages 282–305. Springer, 2018.
- [DMSS24] Pousali Dey, Pratyay Mukherjee, Swagata Sasmal, and Rohit Sinha. Hise: Hierarchical (threshold) symmetric-key encryption. Cryptology ePrint Archive, Paper 2024/158, 2024. <https://eprint.iacr.org/2024/158>.
- [DT06] Ivan Damgård and Rune Thorbek. Linear integer secret sharing and distributed exponentiation. In *International Workshop on Public Key Cryptography*, pages 75–90. Springer, 2006.
- [ILL89] Russell Impagliazzo, Leonid A Levin, and Michael Luby. Pseudo-random generation from one-way functions. In *Proceedings of the twenty-first annual ACM symposium on Theory of Computing (STOC) 1989*, pages 12–24, 1989.
- [LPR13] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. A toolkit for ring-lwe cryptography. In *Annual international conference on the theory and applications of cryptographic techniques*, pages 35–54. Springer, 2013.
- [LS15] Adeline Langlois and Damien Stehlé. Worst-case to average-case reductions for module lattices. *Designs, Codes and Cryptography*, 75(3):565–599, 2015.
- [LST21] Benoît Libert, Damien Stehlé, and Radu Titu. Adaptively secure distributed prfs from lwe. *Journal of Cryptology*, 34(3):1–49, 2021.
- [LT19] Benoît Libert and Radu Țițiu. Multi-client functional encryption for linear functions in the standard model from lwe. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 520–551. Springer, 2019.
- [MGB⁺24] Florian Le Mouél, Maxime Godon, Renaud Brien, Erwan Beurier, Nora Boulahia-Cuppens, and Frédéric Cuppens. Trustless distributed symmetric-key encryption. *arXiv preprint arXiv:2408.16137*, 2024.
- [Muk20] Pratyay Mukherjee. Adaptively secure threshold symmetric-key encryption. In *International Conference on Cryptology in India*, pages 465–487. Springer, 2020.
- [NPR99] Moni Naor, Benny Pinkas, and Omer Reingold. Distributed pseudo-random functions and kdc. In *International Conference on the Theory and Applications of Cryptographic Techniques*, pages 327–346. Springer, 1999.
- [NR97] Moni Naor and Omer Reingold. Number-theoretic constructions of efficient pseudo-random functions. In *38th Annual Symposium on Foundations of Computer Science, FOCS ’97, Miami Beach, Florida, USA, October 19-22, 1997*, pages 458–467. IEEE Computer Society, 1997.
- [Reg09] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. *Journal of the ACM (JACM)*, 56(6):1–40, 2009.
- [Sha79] Adi Shamir. How to share a secret. *Communications of the ACM*, 22(11):612–613, 1979.
- [Sho94] Peter W Shor. Algorithms for quantum computation: discrete logarithms and factoring. In *Proceedings 35th annual symposium on foundations of computer science*, pages 124–134. Ieee, 1994.
- [SPM24] Sayani Sinha, Sikhar Patranabis, and Debdeep Mukhopadhyay. Efficient quantum-safe distributed prf and applications: Playing dice in a quantum world. In *International Conference on Applied Cryptography and Network Security*, pages 47–78. Springer, 2024.

A Known DPRF Constructions

In this section, we briefly recall the following existing DPRFs: (i) Box-1: DDH-based DPRF, as used in DiSE [AMMR18], (ii) Box2: Combinatorial AES-based DPRF, as used in DiSE [AMMR18], (iii) Box-3: DDH-based adaptive DPRF, as used in adaptive DiSE [Muk20], (iv) Box-4: LWE-based adaptive DPRF, as described in [LST21], (v) Box-5: ThFHE-based DPRF from AES, using *Universal Thresholdizer* (UT) tool of [BGG⁺18].

The DDH-based DPRF [BOX-1]

Let $G = \langle g \rangle$ be a multiplicative cyclic group of prime order p in which the DDH assumption holds and $\mathcal{H} : \{0, 1\}^* \rightarrow G$ be a hash function modelled as a random oracle. Let $\mathcal{P} = \{P_i : i \in [T]\}$ be a set of T parties, and $\mathcal{P}' \subset \mathcal{P}$ denotes a subset of size t .

- **Setup**($1^\lambda, t, T$): Sample a secret $s \xleftarrow{\$} \mathbb{Z}_p$ and then distribute it among T parties using Shamir's secret sharing algorithm such that each P_i gets s_i as its share.
- **PartialEval**($x, P_i, \mathcal{P}'$): Party P_i computes partial evaluation $\mu_i = \mathcal{H}(x)^{s_i}$ using its secret share s_i , and sends it to rest of the $(t - 1)$ parties of $\mathcal{P}'$.
- **FinalEval**($\{\mu_i\}_{P_i \in \mathcal{P}'}, \mathcal{P}'$): Once all t partial evaluations of the parties in $\mathcal{P}'$ are together, final DPRF output is computed as $\prod_{P_i \in \mathcal{P}'} \mu_i^{\ell_{i, \mathcal{P}'}}$, where $\ell_{i, \mathcal{P}'}$ corresponds to Lagrange's coefficient of P_i for group $\mathcal{P}'$.

The AES-based DPRF [BOX-2]

Let $f : \{0, 1\}^\lambda \times \{0, 1\}^* \rightarrow \{0, 1\}^*$ be the AES function. Let $\mathcal{P} = \{P_i : i \in [T]\}$ be a set of T parties, and $\mathcal{P}' \subset \mathcal{P}$ denotes a subset of size t .

- **Setup**($1^\lambda, t, T$): Sample $k_1, \dots, k_d \xleftarrow{\$} \{0, 1\}^\lambda$, where $d = \binom{T}{T-t+1}$. Let $S_1, \dots, S_d$ be d distinct $(T - t + 1)$ -sized subsets of $\mathcal{P}$. Each k_i is given to all the $(T - t + 1)$ parties in S_i , $\forall i \in [d]$. Effectively, each party holds $\binom{T-1}{T-t}$ number of shares.
- **PartialEval**($x, P_i, \mathcal{P}'$): P_i computes $\mu_i = \{f_{k_j}(x) \mid P_i \text{ owns } k_j\}$.
- **FinalEval**($\{\mu_i\}_{P_i \in \mathcal{P}'}, \mathcal{P}'$): Sparse each μ_i as $\{\mu_{i,j} \mid P_i \text{ owns } k_j\}$. From all μ_i for $P_i \in \mathcal{P}'$, extract a set of $\mu_{i,j}$ such that exactly one $\mu_{i,j}$ is there for each $j \in [d]$. We 'xor' all such $\mu_{i,j}$ to get the final evaluation.

The Adaptive DDH-based DPRF [BOX-3]

Let $G = \langle g \rangle$ be a multiplicative cyclic group of prime order p in which the DDH assumption holds and $\mathcal{H} : \{0, 1\}^* \rightarrow G^2$ be a hash function modelled as a random oracle. Let $\mathcal{P} = \{P_i : i \in [T]\}$ be a set of T parties, and $\mathcal{P}' \subset \mathcal{P}$ denotes a subset of size t .

- **Setup**($1^\lambda, t, T$): Sample secret $(u, v) \xleftarrow{\$} \mathbb{Z}_p^2$ and then distribute it among T parties by applying Shamir's secret sharing scheme individually on both u and v , such that each P_i gets (u_i, v_i) as its share.
- **PartialEval**($x, P_i, \mathcal{P}'$): Party P_i computes partial evaluation $\mu_i = w_1^{u_i} w_2^{v_i}$, such that $\mathcal{H}(x) = (w_1, w_2) \in \mathbb{Z}^2$, and sends it to rest of the $(t - 1)$ parties of $\mathcal{P}'$.
- **FinalEval**($\{\mu_i\}_{P_i \in \mathcal{P}'}, \mathcal{P}'$): Once all t partial evaluations of the parties in $\mathcal{P}'$ are together, final DPRF output is computed as $\prod_{P_i \in \mathcal{P}'} \mu_i^{\ell_{i, \mathcal{P}'}}$, where $\ell_{i, \mathcal{P}'}$ corresponds to Lagrange's coefficient of P_i for group $\mathcal{P}'$.

Post-quantum Adaptive LWE-based DPRF [BOX-4]

Let q, q_1, p be three moduli with sufficiently large $\frac{q}{q_1}$ and $\frac{q_1}{p}$ ratio. Let $n, m = \text{poly}(\lambda)$, where λ is the security parameter and $m \geq 2n \log q$. Let $AHF : \{0, 1\}^\ell \rightarrow \{0, 1\}^L$ be a balanced admissible hash function, and $\pi : \mathbb{Z}_{q_1}^m \rightarrow \mathbb{Z}_p^k$ be a deterministic randomness extractor. $\mathcal{P} = \{P_i : i \in [T]\}$ is a set of T parties and $\mathcal{P}' \subset \mathcal{P}$ is a subset of size t .

- **Setup**($1^\lambda, t, T$): Choose random matrices $\mathbf{A}_0 \xleftarrow{\$} \mathbb{Z}_q^{n \times m}$ and $\mathbf{A}_{i,b} \xleftarrow{\$} \mathbb{Z}_q^{n \times m}$, for each $i \in [L]$ and $b \in \{0, 1\}$ such that each $\mathbf{G}^{-1}(\mathbf{A}_{i,b}) \in \mathbb{Z}^{m \times m}$ is $\mathbb{Z}_q$ -invertible. The secret $\mathbf{s} \in \mathbb{Z}_q^n$ is a small-norm vector sampled from a Gaussian distribution. Apply linear integer secret sharing (LISSS) [DT06] to share it among T parties.
- **PartialEval**($x \in \{0, 1\}^\ell, P_i, \mathcal{P}'$): First, $\tilde{x} = AHF(x) \in \{0, 1\}^L$ is computed. Next, input-dependent matrix $\mathbf{A}(\tilde{x}) = \mathbf{A}_0 \cdot \prod_{i=1}^L \mathbf{G}^{-1}(\mathbf{A}_{i, \tilde{x}_i})$ is computed. Then the partial evaluation, computed by party P_i is $\mu_i = \lfloor \mathbf{A}(\tilde{x})^\top \cdot \mathbf{s}_i \rfloor_{q_1} \in \mathbb{Z}_q^m$, where $\mathbf{s}_i$ is the secret share of P_i corresponding to $\mathcal{P}'$.
- **FinalEval**($\{\mu_i\}_{P_i \in \mathcal{P}'}, \mathcal{P}'$): Once all t partial evaluations corresponding to t -sized subset $\mathcal{P}'$ is received, they are combined to get the final DPRF output as follows: $\pi(\lfloor \sum_{i=1}^t c_i \mu_i \rfloor_p)$, where c_i 's are the integer coefficients required for reconstructing secret $\mathbf{s}$ from shares of t parties.

ThFHE-based AES DPRF [BOX-5]

- **Setup:** FHE-encrypt the AES input (i.e., the PRF input). Distribute the FHE-decryption key (i.e., the PRF key) among multiple parties using threshold secret sharing of [DT06].
- **PartialEval:** Each party performs homomorphic evaluation of the AES circuit in the presence of encrypted AES input and evaluation key; then performs partial decryption on the encrypted AES output and returns the partial decryption as partial DPRF evaluation.
- **FinalEval:** With all the parties' partial evaluations combined, final evaluation of DPRF is equivalent to final decryption phase of threshold FHE. Partial descriptions are combined and decoded to retrieve the output of AES evaluation in plaintext.

B (t, T) -Threshold Linear Integer Secret Sharing

A threshold secret sharing scheme is an essential underlying primitive to build a distributed PRF protocol. A (t, T) -threshold secret sharing scheme shares a key k among these T parties in such a way that any t or more parties are able to reconstruct it from their respective shares, while collaboration of less than t parties does not suffice. We choose to use Benaloh-Leichter Linear Integer Secret Sharing Scheme (LISSS) as described in [DT06]. The secret sharing scheme is “linear integer” because key shares can be linearly combined to get the actual secret back in a way that the coefficients of the linear combination are integers. These coefficients used during the reconstruction of the secret are called *recovery coefficients*. Though the original Benaloh-Leichter LISSS shares a scalar secret, it can naturally be extended to share a secret which is either in vector or in polynomial form. As we deal with secrets belonging to $\mathbb{Z}_q^n$ or $\mathcal{R}_{N,q}^d$ in this work, we describe the LISSS in the context of sharing a secret vector $\mathbf{k} \in \mathbb{Z}_q^n$ here. For $\mathbf{K} \in \mathcal{R}_{N,q}^d$, we treat the secret $\mathbf{K}$ as a vector of $d \cdot N$ elements in $\mathbb{Z}_q$, and the secret sharing in similar manner. We describe the secret sharing algorithm for master secret $\mathbf{k} \in \mathbb{Z}_q^n$ over the T -sized set $\mathcal{P} = \{P_i \mid i \in [T]\}$ for (t, T) -threshold setting in Algorithm 1.

Algorithm 1 SecretShare($\mathbf{k}, \mathcal{P} = \{P_i \mid i \in [T]\}, t, T$)

- 1: Let $\tilde{\mathcal{P}} = \{\mathcal{P}' \mid \mathcal{P}' \subset \mathcal{P} \wedge |\mathcal{P}'| = t\}$ be the set of all t -sized subset of $\mathcal{P}$.
 - 2: **for** $\mathcal{P}' \in \tilde{\mathcal{P}}$ **do**
 - 3: $\mathbf{k}_{\mathcal{P}',1} \leftarrow \mathbf{k}$
 - 4: **for** $\text{pid} = 2$ to t **do**
 - 5: $\mathbf{k}_{\mathcal{P}',\text{pid}} \xleftarrow{\$} \mathbb{Z}_q^n$
 - 6: $\mathbf{k}_{\mathcal{P}',1} = \mathbf{k}_{\mathcal{P}',1} + \mathbf{k}_{\mathcal{P}',\text{pid}}$
 - 7: Delete $\mathbf{k}$
 - 8: **return** $\{\mathbf{k}_{\mathcal{P}',\text{pid}}\}_{\mathcal{P}' \in \tilde{\mathcal{P}}, \text{pid} \in [t]}$ $\triangleright$ Reconstruction: $\mathbf{k} = \mathbf{k}_{\mathcal{P}',1} - \sum_{\text{pid}=2}^t \mathbf{k}_{\mathcal{P}',\text{pid}}, \forall \mathcal{P}' \in \tilde{\mathcal{P}}$
-

Size of Secret Shares. After applying (t, T) -threshold secret sharing on $\mathbf{k} \in \mathbb{Z}_q^n$, each party gets $\binom{T-1}{t-1}$ key shares to store. So each party has to store $\binom{T-1}{t-1} \cdot n \cdot \lceil \log_2 q \rceil$ bits in total.

Choice of LISS over Shamir's secret sharing. We use LISS for sharing the PRF secret instead of the well known Shamir's secret sharing [Sha79], since reconstruction of the secret requires only $\{-1, 1\}$ -integer linear combination of the shares in the former case while in the latter case, much larger Lagrange coefficients serve as the recovery coefficients. Hence, the rounding error from each party gets multiplied by the Lagrange coefficients and it would require even larger (practically inefficient) $\frac{q-1}{p}$ ratio in order to eliminate the errors in the final rounding phase.